

```
In [1]: import pandas as pd

# Load the dataset
data = pd.read_excel("data_filtered.xlsx", sheet_name="full_data")

# Remove non-comparable territories / aggregate categories
excluded_countries = [
    "St Barthelemy",
    "St Martin (French part)",
    "Reunion",
    "Martinique",
    "Guadeloupe",
    "Aland Islands",
    "Cyprus (Not otherwise specified)",
    "French West Indies (Not In Use)",
    "Canary Islands",
    "French Guiana"
]

data = data[~data["Domicile_named_country"].isin(excluded_countries)].copy()

# Define treated and synthetic control country groups
eu_real = [
    "Austria", "Belgium", "Bulgaria", "Croatia", "Cyprus (European Union)",
    "Czech Republic", "Denmark", "Estonia", "Finland", "France",
    "Germany", "Greece", "Hungary", "Italy",
    "Latvia", "Lithuania", "Luxembourg", "Malta", "Netherlands",
    "Poland", "Portugal", "Romania", "Slovakia", "Slovenia",
    "Spain", "Sweden"
]

eu_synth = [
    "Australia", "Saudi Arabia", "Russian Federation", "Peru", "Kuwait",
    "Bahrain", "Singapore", "Switzerland", "Hong Kong",
    "United States of America", "Thailand", "Canada", "Mauritius",
    "Qatar", "Norway", "Israel", "Malaysia", "Jordan", "Albania",
    "Serbia", "Ireland", "Jersey", "Macao", "United Arab Emirates",
    "Vietnam [Viet Nam]", "Turkey"
]

# Keep only observations in the treated or synthetic groups
data = data[data["Domicile_named_country"].isin(eu_real + eu_synth)].copy()

# Create indicator variables and group label
data["real_EU"] = data["Domicile_named_country"].isin(eu_real).astype(int)
data["synth_EU"] = data["Domicile_named_country"].isin(eu_synth).astype(int)

data["EU_group"] = "Other"
data.loc[data["Domicile_named_country"].isin(eu_real), "EU_group"] = "Real_EU"
data.loc[data["Domicile_named_country"].isin(eu_synth), "EU_group"] = "Synthetic_EU"

# Basic checks
print(data["Domicile_named_country"].unique())
```

```
print(data["EU_group"].value_counts())
print(data.head())
```

```
['Austria' 'Belgium' 'Bulgaria' 'Denmark' 'Finland' 'France' 'Germany'
 'Greece' 'Hungary' 'Ireland' 'Italy' 'Luxembourg' 'Malta' 'Netherlands'
 'Poland' 'Portugal' 'Romania' 'Spain' 'Sweden' 'Estonia' 'Latvia'
 'Lithuania' 'Croatia' 'Slovenia' 'Czech Republic' 'Slovakia'
 'Cyprus (European Union)' 'Jersey' 'Russian Federation' 'Mauritius'
 'Norway' 'Peru' 'Qatar' 'Saudi Arabia' 'Singapore' 'Switzerland'
 'Thailand' 'United Arab Emirates' 'Turkey' 'United States of America'
 'Vietnam [Viet Nam]' 'Serbia' 'Albania' 'Australia' 'Bahrain' 'Canada'
 'Hong Kong' 'Israel' 'Jordan' 'Kuwait' 'Macao' 'Malaysia']
```

```
EU_group
```

```
Real_EU      260
```

```
Synthetic_EU  260
```

```
Name: count, dtype: int64
```

```
      Year Domicile_named_country Applicants Accepted_applicants \
40  2016                Austria         650             370
41  2017                Austria         590             340
42  2018                Austria         625             335
43  2019                Austria         625             345
44  2020                Austria         550             295
```

```
      Acceptance_rate Applicants_with_no_applications  real_EU  synth_EU \
40  0.5670000000000001                5                1        0
41  0.5710000000000001                0                1        0
42                0.537                0                1        0
43                0.551                0                1        0
44                0.535                0                1        0
```

```
EU_group
```

```
40 Real_EU
```

```
41 Real_EU
```

```
42 Real_EU
```

```
43 Real_EU
```

```
44 Real_EU
```

```
In [2]: import numpy as np
import pandas as pd
from linearmodels.panel import PanelOLS

# -----
# Sanity check
# -----
data["EU_group"] = None
data.loc[data["Domicile_named_country"].isin(eu_real), "EU_group"] = "Real_EU"
data.loc[data["Domicile_named_country"].isin(eu_synth), "EU_group"] = "Synthetic_EU"

print(data["EU_group"].value_counts(dropna=False))

# 260/10 = 26 real European countries
# (excluding Ireland because it did not face the same restrictions,
# and excluding UK for obvious reasons)
# 260/10 = 26 synthetic countries
# (including Ireland as the most plausible counterfactual)
```

```

# -----
# pdata.frame equivalent
# -----
pdata_UCAS = data.set_index(["Domicile_named_country", "Year"]).sort_index()

# -----
# Clean data
# -----
df = pdata_UCAS.reset_index().copy()
df["Domicile_named_country"] = df["Domicile_named_country"].astype(str)
df["Year"] = df["Year"].astype(int)

# -----
# Treatment definition
# -----
df["Post"] = (df["Year"] >= 2021).astype(int)
df["Treated"] = df["Domicile_named_country"].isin(eu_real).astype(int)

# -----
# Pooled control category
# -----
df["treated_factor"] = np.where(
    df["Treated"] == 1,
    df["Domicile_named_country"],
    "CONTROL"
)

# -----
# Saturated interaction terms
# -----
treated_dummies = pd.get_dummies(df["treated_factor"], prefix="tf", dtype=float)
interaction_terms = treated_dummies.mul(df["Post"], axis=0)

control_col = "tf_CONTROL"
if control_col in interaction_terms.columns:
    interaction_terms = interaction_terms.drop(columns=[control_col])

print(interaction_terms.columns.tolist())

# -----
# Panel setup
# -----
df_panel = df.set_index(["Domicile_named_country", "Year"]).sort_index()
interaction_terms.index = df_panel.index

# Dependent variable
y = np.log(df_panel["Applicants"])

# -----
# Model 1: country + year FE
# -----
X1 = interaction_terms.copy()

m_sat_DK = PanelOLS(
    dependent=y,
    exog=X1,

```

```

    entity_effects=True,
    time_effects=True,
    drop_absorbed=True
)

res_sat_DK = m_sat_DK.fit(cov_type="driscoll-kraay")
print(res_sat_DK.summary)

# -----
# Model 2: country FE only
# -----
X2 = interaction_terms.copy()

m_sat_DK1 = PanelOLS(
    dependent=y,
    exog=X2,
    entity_effects=True,
    time_effects=False,
    drop_absorbed=True
)

res_sat_DK1 = m_sat_DK1.fit(cov_type="driscoll-kraay")
print(res_sat_DK1.summary)

# -----
# Model 3: no FE
# -----
X3 = interaction_terms.copy()

m_sat_DK2 = PanelOLS(
    dependent=y,
    exog=X3,
    entity_effects=False,
    time_effects=False,
    drop_absorbed=True
)

res_sat_DK2 = m_sat_DK2.fit(cov_type="driscoll-kraay")
print(res_sat_DK2.summary)

```

```

EU_group
Real_EU      260
Synthetic_EU  260
Name: count, dtype: int64
['tf_Austria', 'tf_Belgium', 'tf_Bulgaria', 'tf_Croatia', 'tf_Cyprus (European Union)', 'tf_Czech Republic', 'tf_Denmark', 'tf_Estonia', 'tf_Finland', 'tf_France', 'tf_Germany', 'tf_Greece', 'tf_Hungary', 'tf_Italy', 'tf_Latvia', 'tf_Lithuania', 'tf_Luxembourg', 'tf_Malta', 'tf_Netherlands', 'tf_Poland', 'tf_Portugal', 'tf_Romania', 'tf_Slovakia', 'tf_Slovenia', 'tf_Spain', 'tf_Sweden']

```

PanelOLS Estimation Summary

```

=====
Dep. Variable:      Applicants    R-squared:      0.8024
Estimator:          PanelOLS      R-squared (Between): -0.0939
No. Observations:    520          R-squared (Within):  0.8023
Date:                Sat, Mar 14 2026  R-squared (Overall): -0.0897
Time:                23:16:04      Log-likelihood      155.07
Cov. Estimator:      Driscoll-Kraay

F-statistic:        67.639
Entities:           52            P-value           0.0000
Avg Obs:            10.0000       Distribution:      F(26,433)
Min Obs:            10.0000
Max Obs:            10.0000
F-statistic (robust): 3.914e+16
P-value             0.0000

Time periods:       10           Distribution:      F(26,433)
Avg Obs:            52.000
Min Obs:            52.000
Max Obs:            52.000

```

Parameter Estimates

```

=====
Parameter  Std. Err.    T-stat    P-value    Lower CI    Upper CI
-----
tf_Austria      -0.7048    0.0825   -8.5443   0.0000   -0.8669   -0.5427
tf_Belgium      -0.7589    0.0981   -7.7354   0.0000   -0.9518   -0.5661
tf_Bulgaria     -2.0434    0.1611  -12.683   0.0000   -2.3601   -1.7268
tf_Croatia      -1.8110    0.1269  -14.272   0.0000   -2.0604   -1.5616
tf_Cyprus (European Union) -1.0584    0.1005  -10.533   0.0000   -1.2559   -0.8609
tf_Czech Republic -1.2935    0.0569  -22.745   0.0000   -1.4052   -1.1817
tf_Denmark      -1.2028    0.1325   -9.0768   0.0000   -1.4632   -0.9423
tf_Estonia      -1.6037    0.1450  -11.061   0.0000   -1.8887   -1.3187
tf_Finland      -1.9126    0.2385   -8.0179   0.0000   -2.3815   -1.4438
tf_France       -0.7534    0.0865   -8.7057   0.0000   -0.9235   -0.5833
tf_Germany      -0.8378    0.1062   -7.8918   0.0000   -1.0464   -0.6291
tf_Greece       -1.0220    0.1067   -9.5746   0.0000   -1.2318   -0.8122
tf_Hungary      -1.2707    0.0800  -15.874   0.0000   -1.4281   -1.1134
tf_Italy        -1.0813    0.1422   -7.6063   0.0000   -1.3607   -0.8019
tf_Latvia       -1.6678    0.1347  -12.384   0.0000   -1.9325   -1.4031
tf_Lithuania    -2.3294    0.2771   -8.4076   0.0000   -2.8739   -1.7848
tf_Luxembourg   -0.6492    0.0688   -9.4398   0.0000   -0.7843   -0.5140
tf_Malta        -0.5179    0.0475  -10.896   0.0000   -0.6114   -0.4245
tf_Netherlands  -0.6912    0.0472  -14.644   0.0000   -0.7840   -0.5985
tf_Poland       -1.8246    0.1561  -11.688   0.0000   -2.1314   -1.5177
tf_Portugal     -1.0954    0.3219   -3.4029   0.0007   -1.7281   -0.4627
tf_Romania      -1.8677    0.1492  -12.521   0.0000   -2.1609   -1.5745
tf_Slovakia     -1.6726    0.1109  -15.084   0.0000   -1.8905   -1.4547
tf_Slovenia     -1.4358    0.1506   -9.5350   0.0000   -1.7318   -1.1399
tf_Spain        -0.6640    0.0744   -8.9283   0.0000   -0.8101   -0.5178
tf_Sweden       -1.1231    0.1705   -6.5879   0.0000   -1.4582   -0.7880

```

F-test for Poolability: 220.55
P-value: 0.0000
Distribution: F(60,433)

Included effects: Entity, Time

PanelOLS Estimation Summary

```

=====
Dep. Variable:      Applicants    R-squared:      0.8209
Estimator:          PanelOLS      R-squared (Between): -0.0800
No. Observations:    520          R-squared (Within):  0.8209
Date:                Sat, Mar 14 2026  R-squared (Overall): -0.0758
Time:                23:16:04      Log-likelihood    106.40
Cov. Estimator:      Driscoll-Kraay

Entities:            52          F-statistic:      77.920
Avg Obs:             10.0000      P-value           0.0000
Min Obs:             10.0000      Distribution:      F(26,442)
Max Obs:             10.0000      F-statistic (robust): -9.856e+17
                                 P-value           1.0000
Time periods:        10          Distribution:      F(26,442)
Avg Obs:             52.000
Min Obs:             52.000
Max Obs:             52.000

```

Parameter Estimates

```

=====
Parameter   Std. Err.   T-stat   P-value   Lower CI   Upper CI
-----
tf_Austria   -0.5252    0.0482   -10.894   0.0000   -0.6200   -0.4305
tf_Belgium   -0.5793    0.0637    -9.0948   0.0000   -0.7045   -0.4541
tf_Bulgaria  -1.8638    0.1257   -14.832   0.0000   -2.1108   -1.6168
tf_Croatia   -1.6314    0.0965   -16.901   0.0000   -1.8211   -1.4417
tf_Cyprus (European Union) -0.8788    0.0652   -13.483   0.0000   -1.0069   -0.7507
tf_Czech Republic -1.1139    0.0421   -26.427   0.0000   -1.1967   -1.0310
tf_Denmark   -1.0232    0.0998   -10.251   0.0000   -1.2193   -0.8270
tf_Estonia   -1.4241    0.1116   -12.761   0.0000   -1.6435   -1.2048
tf_Finland   -1.7330    0.2040    -8.4953   0.0000   -2.1340   -1.3321
tf_France    -0.5738    0.0554   -10.359   0.0000   -0.6826   -0.4649
tf_Germany    -0.6582    0.0719    -9.1561   0.0000   -0.7995   -0.5169
tf_Greece    -0.8424    0.0765   -11.015   0.0000   -0.9927   -0.6921
tf_Hungary    -1.0911    0.0614   -17.776   0.0000   -1.2118   -0.9705
tf_Italy     -0.9017    0.1088    -8.2847   0.0000   -1.1156   -0.6878
tf_Latvia    -1.4882    0.1064   -13.992   0.0000   -1.6972   -1.2792
tf_Lithuania -2.1498    0.2442    -8.8041   0.0000   -2.6297   -1.6699
tf_Luxembourg -0.4696    0.0359   -13.074   0.0000   -0.5402   -0.3990
tf_Malta     -0.3384    0.0425    -7.9575   0.0000   -0.4219   -0.2548
tf_Netherlands -0.5117    0.0177   -28.901   0.0000   -0.5464   -0.4769
tf_Poland    -1.6450    0.1336   -12.315   0.0000   -1.9075   -1.3824
tf_Portugal  -0.9158    0.2987    -3.0666   0.0023   -1.5028   -0.3289
tf_Romania   -1.6881    0.1200   -14.063   0.0000   -1.9240   -1.4522
tf_Slovakia  -1.4930    0.1024   -14.574   0.0000   -1.6943   -1.2917
tf_Slovenia  -1.2562    0.1170   -10.733   0.0000   -1.4863   -1.0262
tf_Spain     -0.4844    0.0672    -7.2110   0.0000   -0.6164   -0.3524
tf_Sweden    -0.9435    0.1375    -6.8644   0.0000   -1.2137   -0.6734

```

=====

F-test for Poolability: 218.17
P-value: 0.0000
Distribution: F(51,442)

Included effects: Entity

PanelOLS Estimation Summary

```
=====
Dep. Variable:      Applicants    R-squared:              0.2013
Estimator:          PanelOLS      R-squared (Between):    0.3375
No. Observations:    520          R-squared (Within):     -28.643
Date:                Sat, Mar 14 2026  R-squared (Overall):    0.2013
Time:                23:16:04      Log-likelihood          -1675.9
Cov. Estimator:      Driscoll-Kraay

Entities:            52          F-statistic:            4.7875
Avg Obs:             10.0000     P-value                 0.0000
Min Obs:             10.0000     Distribution:            F(26,494)
Max Obs:             10.0000     F-statistic (robust):    -2.794e+17
                                P-value                 1.0000
Time periods:        10          Distribution:            F(26,494)
Avg Obs:             52.000
Min Obs:             52.000
Max Obs:             52.000
=====
```

Parameter Estimates

```
=====
Parameter  Std. Err.    T-stat    P-value    Lower CI    Upper CI
-----
tf_Austria      5.8833    0.0327    179.83    0.0000    5.8190    5.9476
tf_Belgium      6.4102    0.0500    128.11    0.0000    6.3119    6.5085
tf_Bulgaria     5.8377    0.0942    62.000    0.0000    5.6527    6.0227
tf_Croatia      4.2720    0.0866    49.322    0.0000    4.1018    4.4421
tf_Cyprus (European Union)  7.0991    0.0287    247.15    0.0000    7.0427    7.1556
tf_Czech Republic  5.9432    0.0403    147.47    0.0000    5.8640    6.0224
tf_Denmark      5.2348    0.0817    64.043    0.0000    5.0742    5.3954
tf_Estonia      4.6070    0.0974    47.319    0.0000    4.4157    4.7982
tf_Finland      5.0270    0.1661    30.261    0.0000    4.7006    5.3534
tf_France       8.0230    0.0517    155.04    0.0000    7.9214    8.1247
tf_Germany      7.4963    0.0552    135.86    0.0000    7.3879    7.6047
tf_Greece       6.9494    0.0695    99.931    0.0000    6.8128    7.0860
tf_Hungary      5.8123    0.0615    94.472    0.0000    5.6914    5.9332
tf_Italy        7.5207    0.0915    82.211    0.0000    7.3410    7.7005
tf_Latvia       4.9159    0.1095    44.900    0.0000    4.7008    5.1310
tf_Lithuania    5.4327    0.2165    25.091    0.0000    5.0073    5.8581
tf_Luxembourg   5.6074    0.0245    229.17    0.0000    5.5594    5.6555
tf_Malta        4.5088    0.0250    180.44    0.0000    4.4597    4.5579
tf_Netherlands  6.3528    0.0164    386.32    0.0000    6.3205    6.3851
tf_Poland       6.7088    0.1338    50.159    0.0000    6.4460    6.9716
tf_Portugal     6.7572    0.2943    22.964    0.0000    6.1790    7.3353
tf_Romania      6.4810    0.1192    54.363    0.0000    6.2468    6.7153
tf_Slovakia     5.2741    0.0899    58.638    0.0000    5.0974    5.4508
tf_Slovenia     3.7365    0.0815    45.833    0.0000    3.5763    3.8967
tf_Spain        7.8086    0.0544    143.42    0.0000    7.7016    7.9156
tf_Sweden       6.2146    0.1177    52.794    0.0000    5.9833    6.4459
=====
```

```

=====

In [4]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from linearmodels.panel import PanelOLS
from scipy import stats
# -----
# Prepare data
# -----
df = df.copy()
df["Domicile_named_country"] = df["Domicile_named_country"].astype(str)
df["Year"] = df["Year"].astype(int)

# Treatment definition
df["Post"] = (df["Year"] >= 2021).astype(int)
df["EU"] = df["Domicile_named_country"].isin(eu_real).astype(int)

# Event time:  $L = t - 2020$ 
df["event_time"] = df["Year"] - 2020
# -----
# Create event-study regressors
# i(event_time, EU, ref = 0)
# -----
event_times = sorted(df["event_time"].unique())

event_cols = {}
for et in event_times:
    if et == 0:
        continue
    colname = f"event_{et}"
    event_cols[colname] = ((df["event_time"] == et) & (df["EU"] == 1)).astype(int)

X_event = pd.DataFrame(event_cols, index=df.index)

df_panel = df.set_index(["Domicile_named_country", "Year"]).sort_index()
X_event.index = df_panel.index
y_event = np.log(df_panel["Applicants"])
m_event = PanelOLS(
    dependent=y_event,
    exog=X_event,
    entity_effects=True,
    time_effects=True,
    drop_absorbed=True
)

res_event = m_event.fit(cov_type="clustered", cluster_entity=True)
print(res_event.summary)

```


PanelOLS Estimation Summary

```
=====
Dep. Variable:      Applicants    R-squared:      0.6479
Estimator:          PanelOLS      R-squared (Between): -0.0838
No. Observations:    520          R-squared (Within):  0.6914
Date:                Sat, Mar 14 2026  R-squared (Overall): -0.0802
Time:                23:27:42      Log-likelihood    4.8509
Cov. Estimator:      Clustered

                               F-statistic:      92.014
Entities:            52          P-value      0.0000
Avg Obs:             10.0000     Distribution:  F(9,450)
Min Obs:             10.0000
Max Obs:             10.0000     F-statistic (robust): 13.680
                               P-value      0.0000
Time periods:        10         Distribution:  F(9,450)
Avg Obs:             52.000
Min Obs:             52.000
Max Obs:             52.000
=====
```

Parameter Estimates

```
=====
      Parameter  Std. Err.    T-stat    P-value    Lower CI    Upper CI
-----
event_-4      0.1239    0.0691    1.7922    0.0738    -0.0120    0.2598
event_-3      0.0789    0.0585    1.3481    0.1783    -0.0361    0.1939
event_-2      0.0782    0.0429    1.8215    0.0692    -0.0062    0.1626
event_-1      0.0574    0.0253    2.2651    0.0240    0.0076    0.1072
event_1     -0.7958    0.0904   -8.8075    0.0000    -0.9733   -0.6182
event_2     -1.1883    0.1134  -10.482    0.0000    -1.4111   -0.9655
event_3     -1.2944    0.1297   -9.9770    0.0000    -1.5494   -1.0394
event_4     -1.3346    0.1354   -9.8578    0.0000    -1.6006   -1.0685
event_5     -1.3742    0.1391   -9.8804    0.0000    -1.6475   -1.1008
=====
```

F-test for Poolability: 163.15

P-value: 0.0000

Distribution: F(60,450)

Included effects: Entity, Time

In [5]:

```
# -----
# Build plotting dataframe
# -----
coef = res_event.params
se = res_event.std_errors

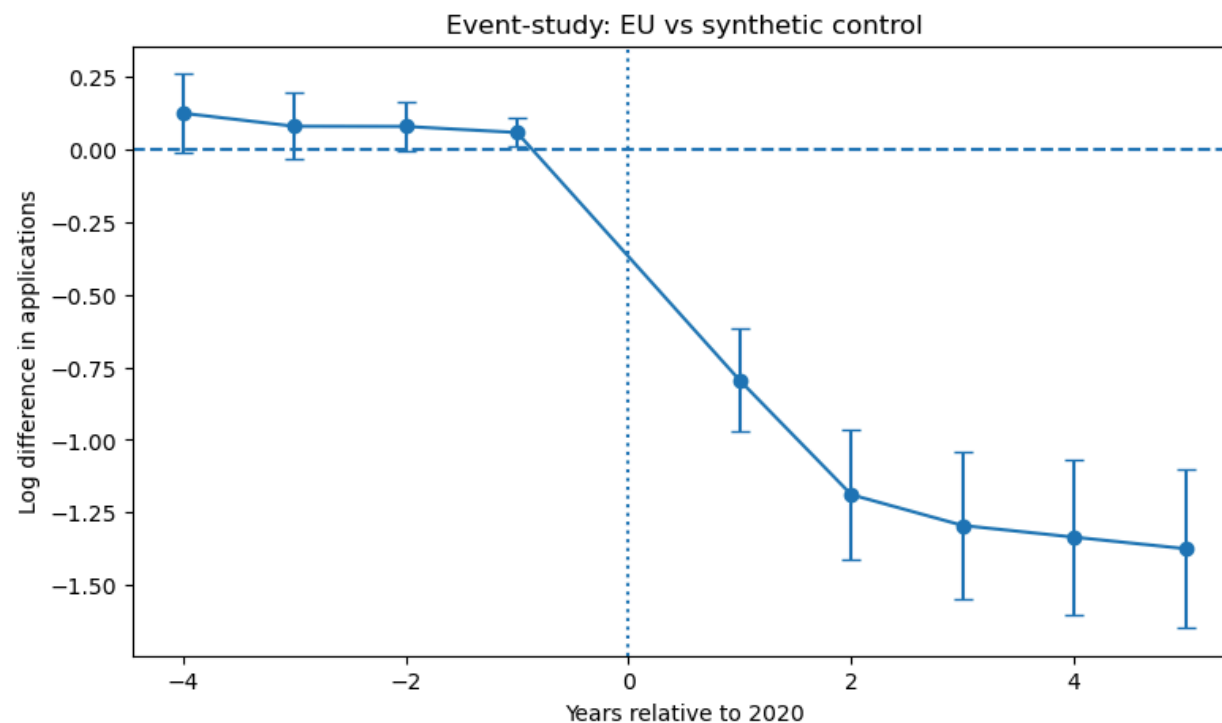
plot_df = pd.DataFrame({
    "term": coef.index,
    "estimate": coef.values,
    "std_error": se.values
})

plot_df["event_time"] = plot_df["term"].str.extract(r'event_(-?\d+)').astype(int)
plot_df["conf_low"] = plot_df["estimate"] - 1.96 * plot_df["std_error"]
plot_df["conf_high"] = plot_df["estimate"] + 1.96 * plot_df["std_error"]
```

```
plot_df = plot_df.sort_values("event_time")
print(plot_df)
```

	term	estimate	std_error	event_time	conf_low	conf_high
0	event_-4	0.123901	0.069133	-4	-0.011599	0.259402
1	event_-3	0.078886	0.058518	-3	-0.035810	0.193582
2	event_-2	0.078230	0.042948	-2	-0.005948	0.162408
3	event_-1	0.057376	0.025330	-1	0.007729	0.107023
4	event_1	-0.795767	0.090352	1	-0.972856	-0.618678
5	event_2	-1.188341	0.113370	2	-1.410546	-0.966135
6	event_3	-1.294385	0.129737	3	-1.548669	-1.040101
7	event_4	-1.334555	0.135381	4	-1.599901	-1.069209
8	event_5	-1.374155	0.139079	5	-1.646750	-1.101560

```
In [6]: plt.figure(figsize=(9, 5))
plt.axhline(0, linestyle="--")
plt.axvline(0, linestyle=":")
plt.errorbar(
    plot_df["event_time"],
    plot_df["estimate"],
    yerr=1.96 * plot_df["std_error"],
    fmt="o-",
    capsize=4
)
plt.xlabel("Years relative to 2020")
plt.ylabel("Log difference in applications")
plt.title("Event-study: EU vs synthetic control")
plt.show()
```



```
In [7]: # -----
# Wald test: pre-trends
# H0: coefficients on event_-4, event_-3, event_-2, event_-1 = 0
# -----
pre_terms = ["event_-4", "event_-3", "event_-2", "event_-1"]
pre_terms = [t for t in pre_terms if t in res_event.params.index]

b = res_event.params[pre_terms].values
V = res_event.cov.loc[pre_terms, pre_terms].values

W = b.T @ np.linalg.inv(V) @ b
df_wald = len(pre_terms)
p_value = 1 - stats.chi2.cdf(W, df_wald)

print("Wald statistic:", W)
print("df:", df_wald)
print("p-value:", p_value)
```

Wald statistic: 6.717711426796582
df: 4
p-value: 0.15157933290685588

```
In [9]: # -----
# Placebo event study: fake Brexit in 2018
# ref = -1
# -----
df["event_time_pl2018"] = df["Year"] - 2018

event_times_pl = sorted(df["event_time_pl2018"].unique())
placebo_cols = {}

for et in event_times_pl:
    if et == -1:
        continue
    colname = f"pl2018_{et}"
    placebo_cols[colname] = ((df["event_time_pl2018"] == et) & (df["EU"] == 1)).astype(int)

X_pl2018 = pd.DataFrame(placebo_cols, index=df.index)

df_panel = df.set_index(["Domicile_named_country", "Year"]).sort_index()
X_pl2018.index = df_panel.index
y_pl2018 = np.log(df_panel["Applicants"])

m_placebo_2018 = PanelOLS(
    dependent=y_pl2018,
    exog=X_pl2018,
    entity_effects=True,
    time_effects=True,
    drop_absorbed=True
)

res_placebo_2018 = m_placebo_2018.fit(cov_type="clustered", cluster_entity=True)
print(res_placebo_2018.summary)
```

PanelOLS Estimation Summary

```
=====
Dep. Variable:      Applicants    R-squared:      0.6479
Estimator:          PanelOLS      R-squared (Between): -0.0961
No. Observations:    520          R-squared (Within):  0.6914
Date:                Sat, Mar 14 2026  R-squared (Overall): -0.0924
Time:                23:29:21      Log-likelihood    4.8509
Cov. Estimator:      Clustered

                               F-statistic:      92.014
Entities:            52           P-value      0.0000
Avg Obs:             10.0000      Distribution:  F(9,450)
Min Obs:             10.0000
Max Obs:             10.0000      F-statistic (robust): 13.680
                               P-value      0.0000
Time periods:        10          Distribution:  F(9,450)
Avg Obs:             52.000
Min Obs:             52.000
Max Obs:             52.000
=====
```

Parameter Estimates

```
=====
Parameter  Std. Err.    T-stat    P-value    Lower CI    Upper CI
-----
pl2018_-2   0.0450    0.0371    1.2136    0.2255    -0.0279    0.1179
pl2018_0   -0.0007    0.0308   -0.0213    0.9830    -0.0612    0.0599
pl2018_1   -0.0215    0.0460   -0.4680    0.6400    -0.1118    0.0688
pl2018_2   -0.0789    0.0585   -1.3481    0.1783    -0.1939    0.0361
pl2018_3   -0.8747    0.1114   -7.8547    0.0000    -1.0935   -0.6558
pl2018_4   -1.2672    0.1265  -10.018    0.0000    -1.5158   -1.0186
pl2018_5   -1.3733    0.1465   -9.3761    0.0000    -1.6611   -1.0854
pl2018_6   -1.4134    0.1507   -9.3809    0.0000    -1.7096   -1.1173
pl2018_7   -1.4530    0.1536   -9.4607    0.0000    -1.7549   -1.1512
=====
```

F-test for Poolability: 163.09

P-value: 0.0000

Distribution: F(60,450)

Included effects: Entity, Time

```
In [10]: coef = res_placebo_2018.params
se = res_placebo_2018.std_errors

plot_df_pl2018 = pd.DataFrame({
    "term": coef.index,
    "estimate": coef.values,
    "std_error": se.values
})

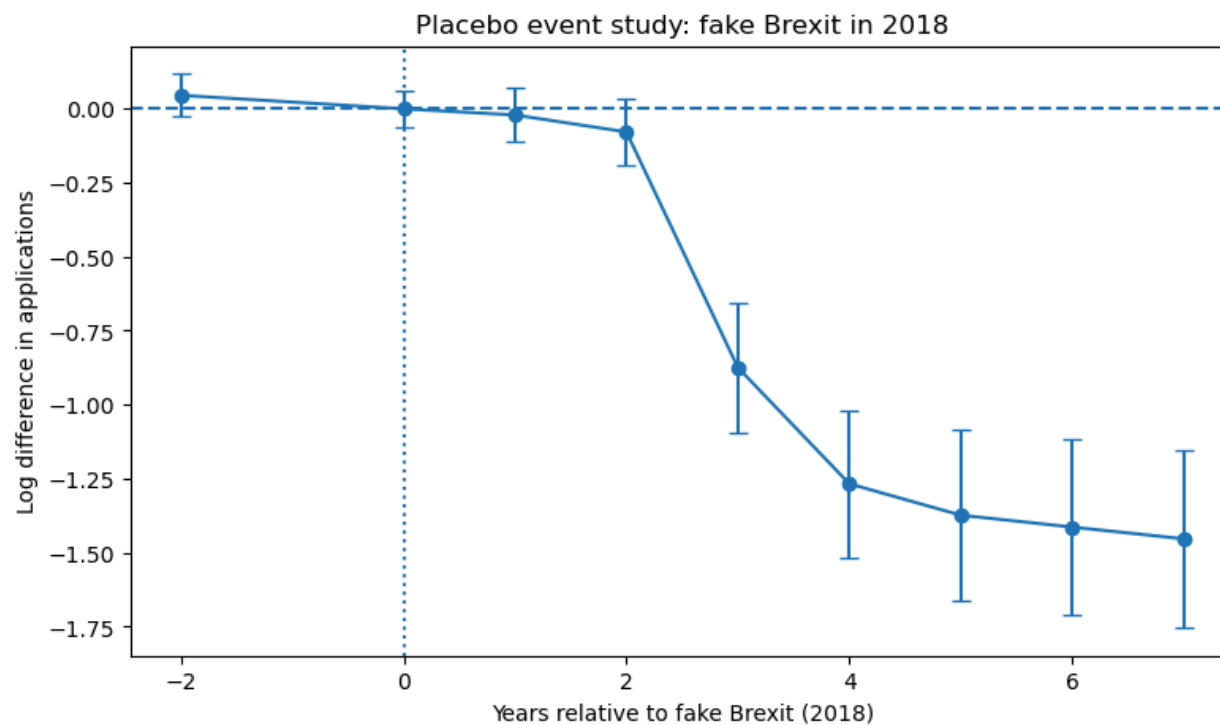
plot_df_pl2018["event_time"] = plot_df_pl2018["term"].str.extract(r'pl2018_(-?\d+)').astype(int)
plot_df_pl2018["conf_low"] = plot_df_pl2018["estimate"] - 1.96 * plot_df_pl2018["std_error"]
plot_df_pl2018["conf_high"] = plot_df_pl2018["estimate"] + 1.96 * plot_df_pl2018["std_error"]
plot_df_pl2018 = plot_df_pl2018.sort_values("event_time")

plt.figure(figsize=(9, 5))
plt.axhline(0, linestyle="--")
```

```

plt.axvline(0, linestyle=":")
plt.errorbar(
    plot_df_pl2018["event_time"],
    plot_df_pl2018["estimate"],
    yerr=1.96 * plot_df_pl2018["std_error"],
    fmt="o-",
    capsize=4
)
plt.xlabel("Years relative to fake Brexit (2018)")
plt.ylabel("Log difference in applications")
plt.title("Placebo event study: fake Brexit in 2018")
plt.show()

```



```

In [11]: # -----
# Leads-only sample
# -----
df_pre = df[df["Year"] < 2021].copy()
df_pre["event_time"] = df_pre["Year"] - 2020

event_times_pre = sorted(df_pre["event_time"].unique())
lead_cols = {}

for et in event_times_pre:
    if et == 0:
        continue
    colname = f"lead_{et}"
    lead_cols[colname] = ((df_pre["event_time"] == et) & (df_pre["EU"] == 1)).astype(int)

```

```
X_leads = pd.DataFrame(lead_cols, index=df_pre.index)

df_pre_panel = df_pre.set_index(["Domicile_named_country", "Year"]).sort_index()
X_leads.index = df_pre_panel.index
y_leads = np.log(df_pre_panel["Applicants"])

m_leads_only = PanelOLS(
    dependent=y_leads,
    exog=X_leads,
    entity_effects=True,
    time_effects=True,
    drop_absorbed=True
)

res_leads_only = m_leads_only.fit(cov_type="clustered", cluster_entity=True)
print(res_leads_only.summary)
```

PanelOLS Estimation Summary

```
=====
Dep. Variable:      Applicants    R-squared:      0.0400
Estimator:          PanelOLS      R-squared (Between): 0.0097
No. Observations:    260          R-squared (Within): -0.0551
Date:                Sat, Mar 14 2026  R-squared (Overall): 0.0097
Time:                23:30:02    Log-likelihood      233.78
Cov. Estimator:      Clustered

                               F-statistic:      2.0848
Entities:              52          P-value      0.0842
Avg Obs:                5.0000      Distribution:  F(4,200)
Min Obs:                5.0000
Max Obs:                5.0000      F-statistic (robust): 1.4928
                               P-value      0.2057
Time periods:          5          Distribution:  F(4,200)
Avg Obs:                52.000
Min Obs:                52.000
Max Obs:                52.000
```

Parameter Estimates

```
=====
               Parameter  Std. Err.    T-stat    P-value    Lower CI    Upper CI
-----
lead_-4      0.1239      0.0733      1.6897      0.0926     -0.0207      0.2685
lead_-3      0.0789      0.0621      1.2710      0.2052     -0.0435      0.2013
lead_-2      0.0782      0.0456      1.7173      0.0875     -0.0116      0.1681
lead_-1      0.0574      0.0269      2.1356      0.0339      0.0044      0.1104
=====
```

```
F-test for Poolability: 458.91
P-value: 0.0000
Distribution: F(55,200)
```

```
Included effects: Entity, Time
```

```
In [12]: coef = res_leads_only.params
         se = res_leads_only.std_errors
```

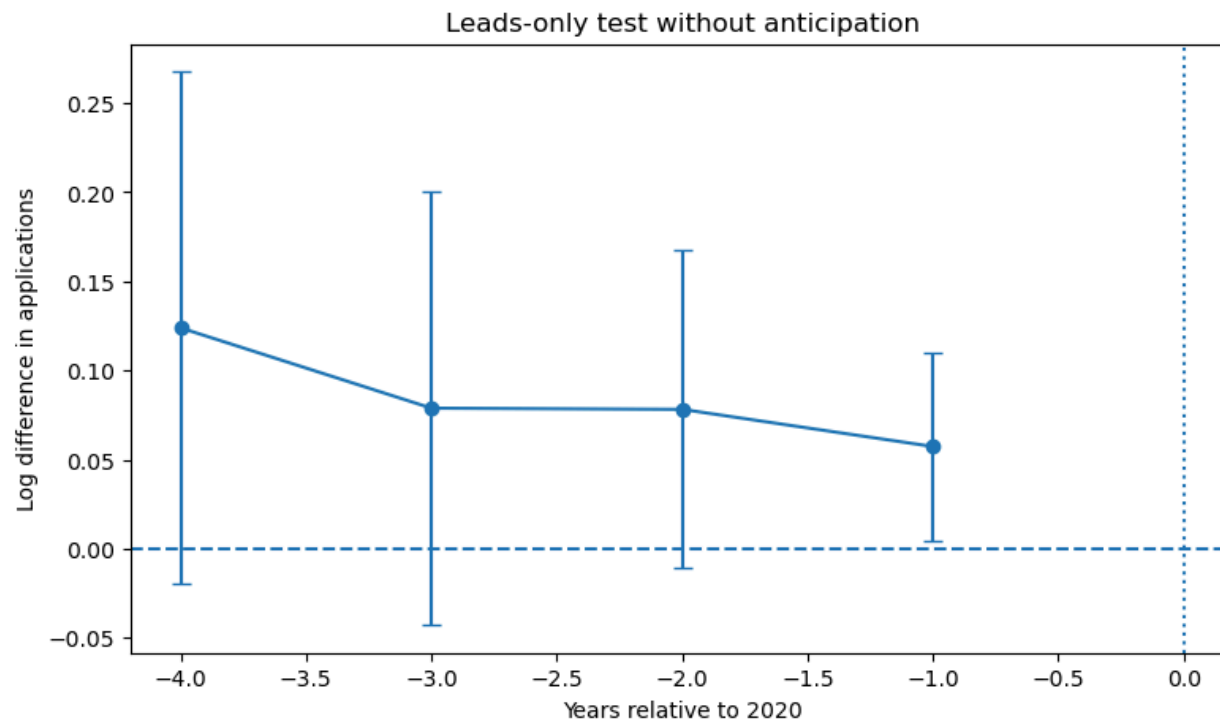
```

plot_df_leads = pd.DataFrame({
    "term": coef.index,
    "estimate": coef.values,
    "std_error": se.values
})

plot_df_leads["event_time"] = plot_df_leads["term"].str.extract(r'lead_(-?\d+)').astype(int)
plot_df_leads["conf_low"] = plot_df_leads["estimate"] - 1.96 * plot_df_leads["std_error"]
plot_df_leads["conf_high"] = plot_df_leads["estimate"] + 1.96 * plot_df_leads["std_error"]
plot_df_leads = plot_df_leads.sort_values("event_time")

plt.figure(figsize=(9, 5))
plt.axhline(0, linestyle="--")
plt.axvline(0, linestyle=":")
plt.errorbar(
    plot_df_leads["event_time"],
    plot_df_leads["estimate"],
    yerr=1.96 * plot_df_leads["std_error"],
    fmt="o-",
    capsize=4
)
plt.xlabel("Years relative to 2020")
plt.ylabel("Log difference in applications")
plt.title("Leads-only test without anticipation")
plt.show()

```



```
In [13]: lead_terms = ["lead_-4", "lead_-3", "lead_-2", "lead_-1"]
lead_terms = [t for t in lead_terms if t in res_leads_only.params.index]

b = res_leads_only.params[lead_terms].values
V = res_leads_only.cov.loc[lead_terms, lead_terms].values

W = b.T @ np.linalg.inv(V) @ b
df_wald = len(lead_terms)
p_value = 1 - stats.chi2.cdf(W, df_wald)

print("Leads-only Wald statistic:", W)
print("df:", df_wald)
print("p-value:", p_value)
```

```
Leads-only Wald statistic: 5.97129904604153
df: 4
p-value: 0.20130195548825525
```

```
In [14]: # -----
# Randomized placebo treatment
# -----
np.random.seed(123)

countries = df["Domicile_named_country"].unique()
fake_EU = np.random.choice(countries, size=len(eu_real), replace=False)

df["EU_placebo"] = df["Domicile_named_country"].isin(fake_EU).astype(int)
df["event_time"] = df["Year"] - 2020

event_times_rand = sorted(df["event_time"].unique())
rand_cols = {}

for et in event_times_rand:
    if et == 0:
        continue
    colname = f"rand_{et}"
    rand_cols[colname] = ((df["event_time"] == et) & (df["EU_placebo"] == 1)).astype(int)

X_rand = pd.DataFrame(rand_cols, index=df.index)

df_panel = df.set_index(["Domicile_named_country", "Year"]).sort_index()
X_rand.index = df_panel.index
y_rand = np.log(df_panel["Applicants"])

m_placebo_random = PanelOLS(
    dependent=y_rand,
    exog=X_rand,
    entity_effects=True,
    time_effects=True,
    drop_absorbed=True
)

res_placebo_random = m_placebo_random.fit(cov_type="clustered", cluster_entity=True)
print(res_placebo_random.summary)
```


PanelOLS Estimation Summary

```
=====
Dep. Variable:      Applicants    R-squared:      0.0064
Estimator:          PanelOLS      R-squared (Between): -0.0039
No. Observations:    520          R-squared (Within):  0.0657
Date:                Sat, Mar 14 2026  R-squared (Overall): -0.0036
Time:                23:31:07      Log-likelihood      -264.90
Cov. Estimator:      Clustered

                               F-statistic:      0.3213
Entities:            52           P-value      0.9679
Avg Obs:             10.0000      Distribution:    F(9,450)
Min Obs:             10.0000
Max Obs:             10.0000      F-statistic (robust): 0.4208
                               P-value      0.9241
Time periods:        10          Distribution:    F(9,450)
Avg Obs:             52.000
Min Obs:             52.000
Max Obs:             52.000
=====
```

Parameter Estimates

```
=====
Parameter  Std. Err.    T-stat    P-value    Lower CI    Upper CI
-----
rand_-4    0.0408    0.0713    0.5723    0.5674    -0.0993    0.1809
rand_-3    0.0477    0.0593    0.8054    0.4210    -0.0687    0.1642
rand_-2    0.0359    0.0442    0.8136    0.4163    -0.0509    0.1228
rand_-1    0.0391    0.0261    1.4992    0.1345    -0.0122    0.0904
rand_1     -0.0363    0.1490   -0.2439    0.8074    -0.3292    0.2565
rand_2     -0.1069    0.2097   -0.5098    0.6104    -0.5191    0.3052
rand_3     -0.1124    0.2319   -0.4848    0.6280    -0.5682    0.3433
rand_4     -0.1032    0.2401   -0.4298    0.6675    -0.5752    0.3687
rand_5     -0.0805    0.2473   -0.3256    0.7449    -0.5665    0.4055
=====
```

F-test for Poolability: 57.772

P-value: 0.0000

Distribution: F(60,450)

Included effects: Entity, Time

```
In [15]: coef = res_placebo_random.params
se = res_placebo_random.std_errors

plot_df_rand = pd.DataFrame({
    "term": coef.index,
    "estimate": coef.values,
    "std_error": se.values
})

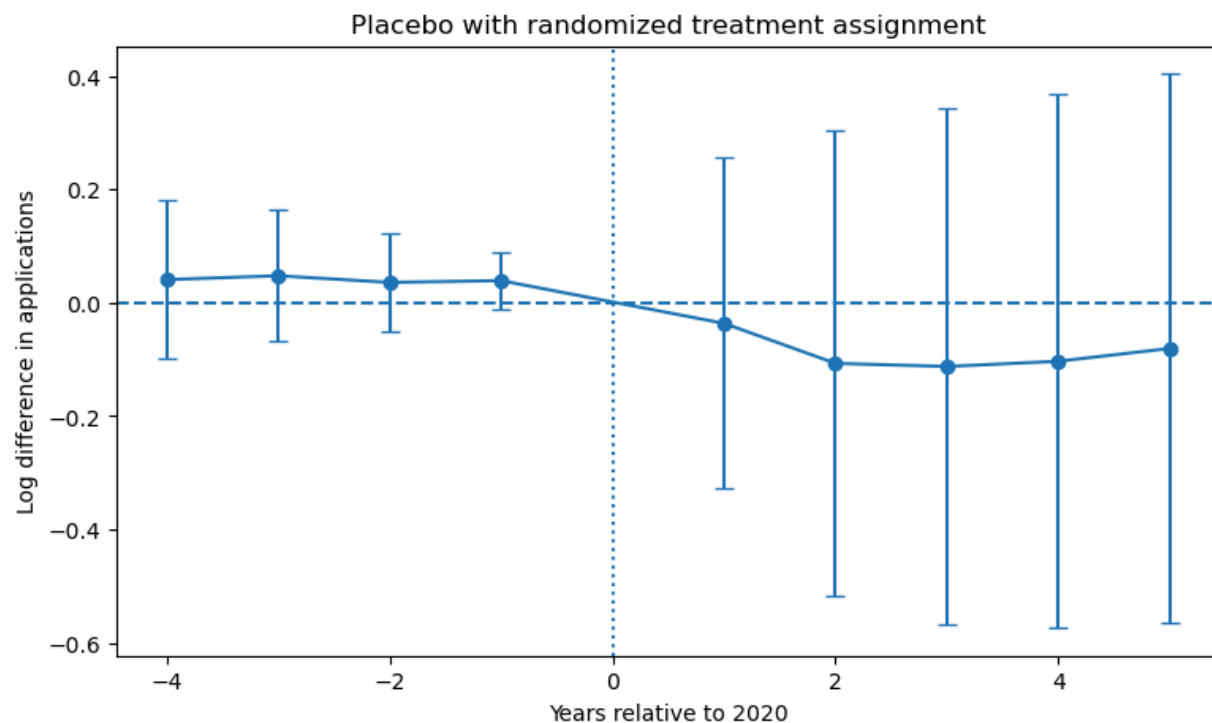
plot_df_rand["event_time"] = plot_df_rand["term"].str.extract(r'rand_(\d+)').astype(int)
plot_df_rand["conf_low"] = plot_df_rand["estimate"] - 1.96 * plot_df_rand["std_error"]
plot_df_rand["conf_high"] = plot_df_rand["estimate"] + 1.96 * plot_df_rand["std_error"]
plot_df_rand = plot_df_rand.sort_values("event_time")

plt.figure(figsize=(9, 5))
plt.axhline(0, linestyle="--")
```

```

plt.axvline(0, linestyle=":")
plt.errorbar(
    plot_df_rand["event_time"],
    plot_df_rand["estimate"],
    yerr=1.96 * plot_df_rand["std_error"],
    fmt="o-",
    capsize=4
)
plt.xlabel("Years relative to 2020")
plt.ylabel("Log difference in applications")
plt.title("Placebo with randomized treatment assignment")
plt.show()

```



```

In [16]: # -----
# Saturated event study by country
# -----
df = pdata_UCAS.reset_index().copy()
df["Domicile_named_country"] = df["Domicile_named_country"].astype(str)
df["Year"] = df["Year"].astype(int)

df["event_time"] = df["Year"] - 2020
df["Treated"] = df["Domicile_named_country"].isin(eu_real)

df["treated_factor"] = np.where(
    df["Treated"],
    df["Domicile_named_country"],
    "CONTROL"
)

```

```
)

print(df["Domicile_named_country"].head())
print(df["treated_factor"].head())
print(sorted(df["treated_factor"].unique()))

0    Albania
1    Albania
2    Albania
3    Albania
4    Albania
Name: Domicile_named_country, dtype: object
0    CONTROL
1    CONTROL
2    CONTROL
3    CONTROL
4    CONTROL
Name: treated_factor, dtype: object
['Austria', 'Belgium', 'Bulgaria', 'CONTROL', 'Croatia', 'Cyprus (European Union)', 'Czech Republic', 'Denmark', 'Estonia', 'Finland', 'France', 'Germany', 'Greece', 'Hungary', 'Italy', 'Latvia', 'Lithuania', 'Luxembourg', 'Malta', 'Netherlands', 'Poland', 'Portugal', 'Romania', 'Slovakia', 'Slovenia', 'Spain', 'Sweden']
```

```
In [17]: sat_cols = {}

treated_countries = sorted([c for c in df["treated_factor"].unique() if c != "CONTROL"])
event_times = sorted(df["event_time"].unique())

for country in treated_countries:
    for et in event_times:
        if et == 0:
            continue
        colname = f"{country}__{et}"
        sat_cols[colname] = ((df["treated_factor"] == country) & (df["event_time"] == et)).astype(int)

X_sat = pd.DataFrame(sat_cols, index=df.index)

df_sat_panel = df.set_index(["Domicile_named_country", "Year"]).sort_index()
X_sat.index = df_sat_panel.index
y_sat = np.log(df_sat_panel["Applicants"])


```

```
In [18]: m_sat_event = PanelOLS(
    dependent=y_sat,
    exog=X_sat,
    entity_effects=True,
    time_effects=True,
    drop_absorbed=True
)

res_sat_event = m_sat_event.fit(cov_type="clustered", cluster_entity=True)
print(res_sat_event.summary)
```

PanelOLS Estimation Summary

```

=====
Dep. Variable:      Applicants    R-squared:              0.8824
Estimator:          PanelOLS      R-squared (Between):    -0.0839
No. Observations:    520          R-squared (Within):     0.8677
Date:                Sat, Mar 14 2026  R-squared (Overall):    -0.0795
Time:                23:32:02      Log-likelihood          289.91
Cov. Estimator:      Clustered

                               F-statistic:          7.2133
Entities:            52           P-value           0.0000
Avg Obs:             10.0000      Distribution:        F(234,225)
Min Obs:             10.0000
Max Obs:             10.0000      F-statistic (robust): -1.361e+15
                               P-value           1.0000
Time periods:        10          Distribution:        F(234,225)
Avg Obs:             52.000
Min Obs:             52.000
Max Obs:             52.000

```

Parameter Estimates

```

=====
Parameter   Std. Err.   T-stat   P-value   Lower CI   Upper CI
-----
Austria__-4    0.2684    0.0795    3.3766    0.0009    0.1118    0.4251
Austria__-3    0.1679    0.0638    2.6322    0.0091    0.0422    0.2935
Austria__-2    0.1775    0.0473    3.7545    0.0002    0.0844    0.2707
Austria__-1    0.1624    0.0291    5.5850    0.0000    0.1051    0.2196
Austria__1     -0.3588    0.0405   -8.8679    0.0000   -0.4385   -0.2791
Austria__2     -0.5237    0.0461  -11.366    0.0000   -0.6145   -0.4329
Austria__3     -0.6294    0.0717   -8.7761    0.0000   -0.7707   -0.4881
Austria__4     -0.5681    0.0933   -6.0892    0.0000   -0.7520   -0.3843
Austria__5     -0.6678    0.1100   -6.0721    0.0000   -0.8846   -0.4511
Belgium__-4     0.2033    0.0795    2.5575    0.0112    0.0467    0.3600
Belgium__-3     0.1728    0.0638    2.7098    0.0073    0.0471    0.2985
Belgium__-2     0.1294    0.0473    2.7359    0.0067    0.0362    0.2226
Belgium__-1     0.0634    0.0291    2.1799    0.0303    0.0061    0.1207
Belgium__1     -0.3990    0.0405   -9.8625    0.0000   -0.4788   -0.3193
Belgium__2     -0.6164    0.0461  -13.379    0.0000   -0.7072   -0.5256
Belgium__3     -0.7007    0.0717   -9.7710    0.0000   -0.8421   -0.5594
Belgium__4     -0.7362    0.0933   -7.8905    0.0000   -0.9200   -0.5523
Belgium__5     -0.7734    0.1100   -7.0320    0.0000   -0.9901   -0.5567
Bulgaria__-4     0.3635    0.0795    4.5729    0.0000    0.2069    0.5202
Bulgaria__-3     0.2551    0.0638    4.0005    0.0001    0.1295    0.3808
Bulgaria__-2     0.2072    0.0473    4.3812    0.0000    0.1140    0.3004
Bulgaria__-1     0.1258    0.0291    4.3286    0.0000    0.0685    0.1831
Bulgaria__1    -1.3572    0.0405  -33.544    0.0000   -1.4370   -1.2775
Bulgaria__2    -1.9612    0.0461  -42.568    0.0000   -2.0520   -1.8704
Bulgaria__3    -1.9021    0.0717  -26.523    0.0000   -2.0435   -1.7608
Bulgaria__4    -2.0222    0.0933  -21.674    0.0000   -2.2060   -1.8383
Bulgaria__5    -2.0226    0.1100  -18.390    0.0000   -2.2393   -1.8059
Croatia__-4     0.1562    0.0795    1.9647    0.0507   -0.0005    0.3129
Croatia__-3     0.0397    0.0638    0.6221    0.5345   -0.0860    0.1653
Croatia__-2     0.1815    0.0473    3.8377    0.0002    0.0883    0.2747
Croatia__-1     0.0623    0.0291    2.1432    0.0332    0.0050    0.1196
Croatia__1     -1.3197    0.0405  -32.617    0.0000   -1.3995   -1.2400
Croatia__2     -1.6683    0.0461  -36.210    0.0000   -1.7591   -1.5775

```

Croatia__3	-1.9262	0.0717	-26.858	0.0000	-2.0675	-1.7849
Croatia__4	-1.7817	0.0933	-19.097	0.0000	-1.9656	-1.5979
Croatia__5	-1.9195	0.1100	-17.453	0.0000	-2.1362	-1.7028
Cyprus (European Union)__-4	0.2692	0.0795	3.3857	0.0008	0.1125	0.4258
Cyprus (European Union)__-3	0.3287	0.0638	5.1539	0.0000	0.2030	0.4543
Cyprus (European Union)__-2	0.1011	0.0473	2.1379	0.0336	0.0079	0.1943
Cyprus (European Union)__-1	0.0532	0.0291	1.8293	0.0687	-0.0041	0.1105
Cyprus (European Union)__1	-0.7403	0.0405	-18.296	0.0000	-0.8200	-0.6606
Cyprus (European Union)__2	-0.8995	0.0461	-19.523	0.0000	-0.9903	-0.8087
Cyprus (European Union)__3	-0.9219	0.0717	-12.855	0.0000	-1.0632	-0.7806
Cyprus (European Union)__4	-0.9690	0.0933	-10.386	0.0000	-1.1529	-0.7852
Cyprus (European Union)__5	-1.0093	0.1100	-9.1772	0.0000	-1.2260	-0.7926
Czech Republic__-4	-0.0674	0.0795	-0.8477	0.3975	-0.2241	0.0893
Czech Republic__-3	-0.0892	0.0638	-1.3990	0.1632	-0.2149	0.0365
Czech Republic__-2	-0.1326	0.0473	-2.8043	0.0055	-0.2258	-0.0394
Czech Republic__-1	-0.0692	0.0291	-2.3790	0.0182	-0.1264	-0.0119
Czech Republic__1	-1.1069	0.0405	-27.358	0.0000	-1.1867	-1.0272
Czech Republic__2	-1.3991	0.0461	-30.368	0.0000	-1.4899	-1.3084
Czech Republic__3	-1.4339	0.0717	-19.994	0.0000	-1.5752	-1.2926
Czech Republic__4	-1.4166	0.0933	-15.183	0.0000	-1.6004	-1.2327
Czech Republic__5	-1.4692	0.1100	-13.358	0.0000	-1.6859	-1.2525
Denmark__-4	0.2025	0.0795	2.5469	0.0115	0.0458	0.3591
Denmark__-3	0.2457	0.0638	3.8529	0.0002	0.1200	0.3714
Denmark__-2	0.2069	0.0473	4.3752	0.0000	0.1137	0.3001
Denmark__-1	0.1547	0.0291	5.3205	0.0000	0.0974	0.2119
Denmark__1	-0.7459	0.0405	-18.436	0.0000	-0.8257	-0.6662
Denmark__2	-0.8958	0.0461	-19.442	0.0000	-0.9865	-0.8050
Denmark__3	-1.1653	0.0717	-16.249	0.0000	-1.3067	-1.0240
Denmark__4	-1.2674	0.0933	-13.584	0.0000	-1.4513	-1.0836
Denmark__5	-1.1297	0.1100	-10.271	0.0000	-1.3464	-0.9130
Estonia__-4	0.2627	0.0795	3.3038	0.0011	0.1060	0.4193
Estonia__-3	0.1224	0.0638	1.9185	0.0563	-0.0033	0.2480
Estonia__-2	0.0371	0.0473	0.7852	0.4332	-0.0561	0.1303
Estonia__-1	0.0592	0.0291	2.0370	0.0428	0.0019	0.1165
Estonia__1	-1.1026	0.0405	-27.251	0.0000	-1.1823	-1.0229
Estonia__2	-1.4512	0.0461	-31.497	0.0000	-1.5420	-1.3604
Estonia__3	-1.5860	0.0717	-22.115	0.0000	-1.7273	-1.4447
Estonia__4	-1.7069	0.0933	-18.295	0.0000	-1.8908	-1.5231
Estonia__5	-1.6905	0.1100	-15.371	0.0000	-1.9073	-1.4738
Finland__-4	0.4260	0.0795	5.3591	0.0000	0.2694	0.5827
Finland__-3	0.2788	0.0638	4.3723	0.0000	0.1532	0.4045
Finland__-2	0.2810	0.0473	5.9431	0.0000	0.1879	0.3742
Finland__-1	0.1323	0.0291	4.5518	0.0000	0.0750	0.1896
Finland__1	-1.1244	0.0405	-27.789	0.0000	-1.2041	-1.0446
Finland__2	-1.4863	0.0461	-32.261	0.0000	-1.5771	-1.3956
Finland__3	-1.8362	0.0717	-25.604	0.0000	-1.9775	-1.6949
Finland__4	-1.9637	0.0933	-21.047	0.0000	-2.1475	-1.7798
Finland__5	-2.0343	0.1100	-18.497	0.0000	-2.2510	-1.8176
France__-4	0.1246	0.0795	1.5674	0.1184	-0.0321	0.2813
France__-3	0.1117	0.0638	1.7509	0.0813	-0.0140	0.2373
France__-2	0.0581	0.0473	1.2293	0.2202	-0.0351	0.1513
France__-1	0.0768	0.0291	2.6435	0.0088	0.0196	0.1341
France__1	-0.4200	0.0405	-10.380	0.0000	-0.4997	-0.3402
France__2	-0.6635	0.0461	-14.400	0.0000	-0.7542	-0.5727
France__3	-0.7263	0.0717	-10.127	0.0000	-0.8676	-0.5850
France__4	-0.7770	0.0933	-8.3281	0.0000	-0.9609	-0.5932

France__5	-0.8089	0.1100	-7.3549	0.0000	-1.0256	-0.5922
Germany__-4	0.2498	0.0795	3.1422	0.0019	0.0931	0.4065
Germany__-3	0.1559	0.0638	2.4450	0.0153	0.0303	0.2816
Germany__-2	0.1589	0.0473	3.3605	0.0009	0.0657	0.2521
Germany__-1	0.0588	0.0291	2.0241	0.0441	0.0016	0.1161
Germany__1	-0.4706	0.0405	-11.631	0.0000	-0.5504	-0.3909
Germany__2	-0.6717	0.0461	-14.580	0.0000	-0.7625	-0.5809
Germany__3	-0.7337	0.0717	-10.231	0.0000	-0.8751	-0.5924
Germany__4	-0.8451	0.0933	-9.0581	0.0000	-1.0290	-0.6613
Germany__5	-0.8442	0.1100	-7.6754	0.0000	-1.0609	-0.6274
Greece__-4	0.1408	0.0795	1.7707	0.0780	-0.0159	0.2974
Greece__-3	0.1330	0.0638	2.0850	0.0382	0.0073	0.2586
Greece__-2	0.0768	0.0473	1.6246	0.1057	-0.0164	0.1700
Greece__-1	0.0492	0.0291	1.6929	0.0919	-0.0081	0.1065
Greece__1	-0.6706	0.0405	-16.574	0.0000	-0.7503	-0.5909
Greece__2	-0.8975	0.0461	-19.480	0.0000	-0.9883	-0.8067
Greece__3	-0.9322	0.0717	-12.999	0.0000	-1.0736	-0.7909
Greece__4	-1.0391	0.0933	-11.137	0.0000	-1.2229	-0.8552
Greece__5	-1.1709	0.1100	-10.647	0.0000	-1.3877	-0.9542
Hungary__-4	-0.0558	0.0795	-0.7019	0.4835	-0.2125	0.1009
Hungary__-3	-0.0865	0.0638	-1.3561	0.1764	-0.2122	0.0392
Hungary__-2	-0.0557	0.0473	-1.1768	0.2405	-0.1488	0.0375
Hungary__-1	-0.0168	0.0291	-0.5770	0.5645	-0.0741	0.0405
Hungary__1	-0.9466	0.0405	-23.395	0.0000	-1.0263	-0.8669
Hungary__2	-1.3964	0.0461	-30.309	0.0000	-1.4872	-1.3056
Hungary__3	-1.3832	0.0717	-19.287	0.0000	-1.5245	-1.2418
Hungary__4	-1.3322	0.0933	-14.279	0.0000	-1.5161	-1.1484
Hungary__5	-1.5100	0.1100	-13.730	0.0000	-1.7267	-1.2933
Italy__-4	0.2947	0.0795	3.7068	0.0003	0.1380	0.4513
Italy__-3	0.1908	0.0638	2.9915	0.0031	0.0651	0.3164
Italy__-2	0.1161	0.0473	2.4558	0.0148	0.0229	0.2093
Italy__-1	0.1396	0.0291	4.8037	0.0000	0.0824	0.1969
Italy__1	-0.5839	0.0405	-14.431	0.0000	-0.6636	-0.5042
Italy__2	-0.8587	0.0461	-18.639	0.0000	-0.9495	-0.7679
Italy__3	-0.9563	0.0717	-13.335	0.0000	-1.0977	-0.8150
Italy__4	-1.1365	0.0933	-12.181	0.0000	-1.3204	-0.9527
Italy__5	-1.1298	0.1100	-10.273	0.0000	-1.3465	-0.9131
Latvia__-4	-0.0107	0.0795	-0.1350	0.8927	-0.1674	0.1459
Latvia__-3	-0.0145	0.0638	-0.2267	0.8208	-0.1401	0.1112
Latvia__-2	-0.0882	0.0473	-1.8643	0.0636	-0.1813	0.0050
Latvia__-1	-0.0443	0.0291	-1.5225	0.1293	-0.1015	0.0130
Latvia__1	-1.1514	0.0405	-28.457	0.0000	-1.2311	-1.0717
Latvia__2	-1.7384	0.0461	-37.731	0.0000	-1.8292	-1.6476
Latvia__3	-1.8957	0.0717	-26.434	0.0000	-2.0370	-1.7544
Latvia__4	-1.8220	0.0933	-19.529	0.0000	-2.0059	-1.6382
Latvia__5	-1.8890	0.1100	-17.176	0.0000	-2.1058	-1.6723
Lithuania__-4	0.2021	0.0795	2.5428	0.0117	0.0455	0.3588
Lithuania__-3	0.2660	0.0638	4.1716	0.0000	0.1404	0.3917
Lithuania__-2	0.2467	0.0473	5.2175	0.0000	0.1535	0.3399
Lithuania__-1	0.1583	0.0291	5.4467	0.0000	0.1011	0.2156
Lithuania__1	-1.2165	0.0405	-30.066	0.0000	-1.2962	-1.1368
Lithuania__2	-2.1401	0.0461	-46.450	0.0000	-2.2309	-2.0493
Lithuania__3	-2.6023	0.0717	-36.286	0.0000	-2.7436	-2.4610
Lithuania__4	-2.5792	0.0933	-27.644	0.0000	-2.7630	-2.3953
Lithuania__5	-2.2356	0.1100	-20.327	0.0000	-2.4523	-2.0189
Luxembourg__-4	0.1485	0.0795	1.8673	0.0632	-0.0082	0.3051

Luxembourg__-3	0.1675	0.0638	2.6258	0.0092	0.0418	0.2931
Luxembourg__-2	0.1527	0.0473	3.2283	0.0014	0.0595	0.2458
Luxembourg__-1	0.0583	0.0291	2.0066	0.0460	0.0010	0.1156
Luxembourg__1	-0.3607	0.0405	-8.9157	0.0000	-0.4405	-0.2810
Luxembourg__2	-0.5813	0.0461	-12.616	0.0000	-0.6720	-0.4905
Luxembourg__3	-0.6160	0.0717	-8.5896	0.0000	-0.7573	-0.4747
Luxembourg__4	-0.6257	0.0933	-6.7064	0.0000	-0.8096	-0.4418
Luxembourg__5	-0.5352	0.1100	-4.8664	0.0000	-0.7519	-0.3185
Malta__-4	0.0636	0.0795	0.8005	0.4242	-0.0930	0.2203
Malta__-3	-0.1537	0.0638	-2.4094	0.0168	-0.2793	-0.0280
Malta__-2	0.0861	0.0473	1.8203	0.0700	-0.0071	0.1793
Malta__-1	-0.0032	0.0291	-0.1107	0.9119	-0.0605	0.0541
Malta__1	-0.4042	0.0405	-9.9891	0.0000	-0.4839	-0.3244
Malta__2	-0.5191	0.0461	-11.268	0.0000	-0.6099	-0.4283
Malta__3	-0.4485	0.0717	-6.2541	0.0000	-0.5898	-0.3072
Malta__4	-0.6207	0.0933	-6.6531	0.0000	-0.8046	-0.4369
Malta__5	-0.6044	0.1100	-5.4950	0.0000	-0.8211	-0.3876
Netherlands__-4	0.1279	0.0795	1.6089	0.1090	-0.0287	0.2846
Netherlands__-3	0.1294	0.0638	2.0292	0.0436	0.0037	0.2551
Netherlands__-2	0.0551	0.0473	1.1646	0.2454	-0.0381	0.1483
Netherlands__-1	0.1170	0.0291	4.0263	0.0001	0.0598	0.1743
Netherlands__1	-0.4663	0.0405	-11.526	0.0000	-0.5461	-0.3866
Netherlands__2	-0.6481	0.0461	-14.066	0.0000	-0.7388	-0.5573
Netherlands__3	-0.5867	0.0717	-8.1804	0.0000	-0.7280	-0.4453
Netherlands__4	-0.6217	0.0933	-6.6632	0.0000	-0.8055	-0.4378
Netherlands__5	-0.7040	0.1100	-6.4014	0.0000	-0.9208	-0.4873
Poland__-4	-0.1342	0.0795	-1.6878	0.0928	-0.2908	0.0225
Poland__-3	-0.1431	0.0638	-2.2444	0.0258	-0.2688	-0.0175
Poland__-2	-0.0475	0.0473	-1.0035	0.3167	-0.1406	0.0457
Poland__-1	-0.0672	0.0291	-2.3113	0.0217	-0.1245	-0.0099
Poland__1	-1.3300	0.0405	-32.871	0.0000	-1.4097	-1.2503
Poland__2	-1.8810	0.0461	-40.827	0.0000	-1.9718	-1.7902
Poland__3	-2.0171	0.0717	-28.127	0.0000	-2.1585	-1.8758
Poland__4	-2.1175	0.0933	-22.695	0.0000	-2.3013	-1.9336
Poland__5	-2.1691	0.1100	-19.722	0.0000	-2.3858	-1.9523
Portugal__-4	-0.4262	0.0795	-5.3608	0.0000	-0.5828	-0.2695
Portugal__-3	-0.2519	0.0638	-3.9503	0.0001	-0.3776	-0.1263
Portugal__-2	-0.0321	0.0473	-0.6786	0.4981	-0.1253	0.0611
Portugal__-1	0.0230	0.0291	0.7898	0.4305	-0.0343	0.0802
Portugal__1	0.1604	0.0405	3.9633	0.0001	0.0806	0.2401
Portugal__2	-1.4735	0.0461	-31.982	0.0000	-1.5643	-1.3827
Portugal__3	-1.6186	0.0717	-22.569	0.0000	-1.7599	-1.4773
Portugal__4	-1.6536	0.0933	-17.723	0.0000	-1.8375	-1.4697
Portugal__5	-1.5791	0.1100	-14.358	0.0000	-1.7958	-1.3624
Romania__-4	0.0974	0.0795	1.2247	0.2220	-0.0593	0.2540
Romania__-3	-0.0536	0.0638	-0.8405	0.4015	-0.1793	0.0721
Romania__-2	-0.0412	0.0473	-0.8712	0.3846	-0.1344	0.0520
Romania__-1	-0.0078	0.0291	-0.2685	0.7885	-0.0651	0.0495
Romania__1	-1.2811	0.0405	-31.663	0.0000	-1.3608	-1.2014
Romania__2	-1.9353	0.0461	-42.005	0.0000	-2.0261	-1.8445
Romania__3	-2.0036	0.0717	-27.939	0.0000	-2.1450	-1.8623
Romania__4	-1.9797	0.0933	-21.219	0.0000	-2.1636	-1.7959
Romania__5	-2.1440	0.1100	-19.494	0.0000	-2.3607	-1.9273
Slovakia__-4	-0.1193	0.0795	-1.5012	0.1347	-0.2760	0.0373
Slovakia__-3	-0.2934	0.0638	-4.6010	0.0000	-0.4191	-0.1677
Slovakia__-2	-0.1893	0.0473	-4.0022	0.0001	-0.2824	-0.0961

Slovakia__-1	0.0101	0.0291	0.3464	0.7294	-0.0472	0.0674
Slovakia__1	-1.3965	0.0405	-34.515	0.0000	-1.4762	-1.3168
Slovakia__2	-1.8355	0.0461	-39.838	0.0000	-1.9262	-1.7447
Slovakia__3	-1.7199	0.0717	-23.983	0.0000	-1.8613	-1.5786
Slovakia__4	-1.9943	0.0933	-21.375	0.0000	-2.1782	-1.8105
Slovakia__5	-2.0087	0.1100	-18.264	0.0000	-2.2254	-1.7920
Slovenia__-4	0.3165	0.0795	3.9811	0.0001	0.1598	0.4732
Slovenia__-3	0.3445	0.0638	5.4023	0.0000	0.2188	0.4702
Slovenia__-2	0.2966	0.0473	6.2716	0.0000	0.2034	0.3898
Slovenia__-1	0.1478	0.0291	5.0860	0.0000	0.0906	0.2051
Slovenia__1	-0.8738	0.0405	-21.595	0.0000	-0.9535	-0.7940
Slovenia__2	-1.1353	0.0461	-24.642	0.0000	-1.2261	-1.0445
Slovenia__3	-1.4214	0.0717	-19.820	0.0000	-1.5627	-1.2801
Slovenia__4	-1.0744	0.0933	-11.516	0.0000	-1.2582	-0.8905
Slovenia__5	-1.5688	0.1100	-14.265	0.0000	-1.7856	-1.3521
Spain__-4	-0.1720	0.0795	-2.1632	0.0316	-0.3286	-0.0153
Spain__-3	-0.1476	0.0638	-2.3151	0.0215	-0.2733	-0.0220
Spain__-2	-0.0869	0.0473	-1.8380	0.0674	-0.1801	0.0063
Spain__-1	-0.0361	0.0291	-1.2417	0.2156	-0.0934	0.0212
Spain__1	-0.4582	0.0405	-11.325	0.0000	-0.5380	-0.3785
Spain__2	-0.7480	0.0461	-16.234	0.0000	-0.8388	-0.6572
Spain__3	-0.8590	0.0717	-11.978	0.0000	-1.0004	-0.7177
Spain__4	-0.8405	0.0933	-9.0085	0.0000	-1.0243	-0.6566
Spain__5	-0.8568	0.1100	-7.7900	0.0000	-1.0735	-0.6400
Sweden__-4	0.2893	0.0795	3.6392	0.0003	0.1327	0.4460
Sweden__-3	0.1748	0.0638	2.7406	0.0066	0.0491	0.3004
Sweden__-2	0.1385	0.0473	2.9284	0.0038	0.0453	0.2317
Sweden__-1	0.0839	0.0291	2.8866	0.0043	0.0266	0.1412
Sweden__1	-0.5650	0.0405	-13.964	0.0000	-0.6447	-0.4853
Sweden__2	-0.8722	0.0461	-18.931	0.0000	-0.9630	-0.7814
Sweden__3	-1.0315	0.0717	-14.383	0.0000	-1.1728	-0.8902
Sweden__4	-1.2073	0.0933	-12.940	0.0000	-1.3912	-1.0235
Sweden__5	-1.2531	0.1100	-11.394	0.0000	-1.4698	-1.0364

=====

F-test for Poolability: 151.20

P-value: 0.0000

Distribution: F(60,225)

Included effects: Entity, Time

```
In [19]: plot_sat_df = pd.DataFrame({
    "term": res_sat_event.params.index,
    "estimate": res_sat_event.params.values,
    "std_error": res_sat_event.std_errors.values
})

plot_sat_df[["country", "event_time"]] = plot_sat_df["term"].str.extract(r"^(.*)__(-?\d+)$")
plot_sat_df["event_time"] = plot_sat_df["event_time"].astype(int)
plot_sat_df["conf_low"] = plot_sat_df["estimate"] - 1.96 * plot_sat_df["std_error"]
plot_sat_df["conf_high"] = plot_sat_df["estimate"] + 1.96 * plot_sat_df["std_error"]

plot_sat_df = plot_sat_df.sort_values(["country", "event_time"])
print(plot_sat_df.head())
```


	term	estimate	std_error	country	event_time	conf_low	conf_high
0	Austria__-4	0.268438	0.079500	Austria	-4	0.112618	0.424258
1	Austria__-3	0.167863	0.063773	Austria	-3	0.042868	0.292858
2	Austria__-2	0.177544	0.047288	Austria	-2	0.084859	0.270229
3	Austria__-1	0.162355	0.029070	Austria	-1	0.105378	0.219331
4	Austria__1	-0.358804	0.040461	Austria	1	-0.438107	-0.279500

```
In [20]: countries = plot_sat_df["country"].unique()
n = len(countries)
ncols = 4
nrows = int(np.ceil(n / ncols))

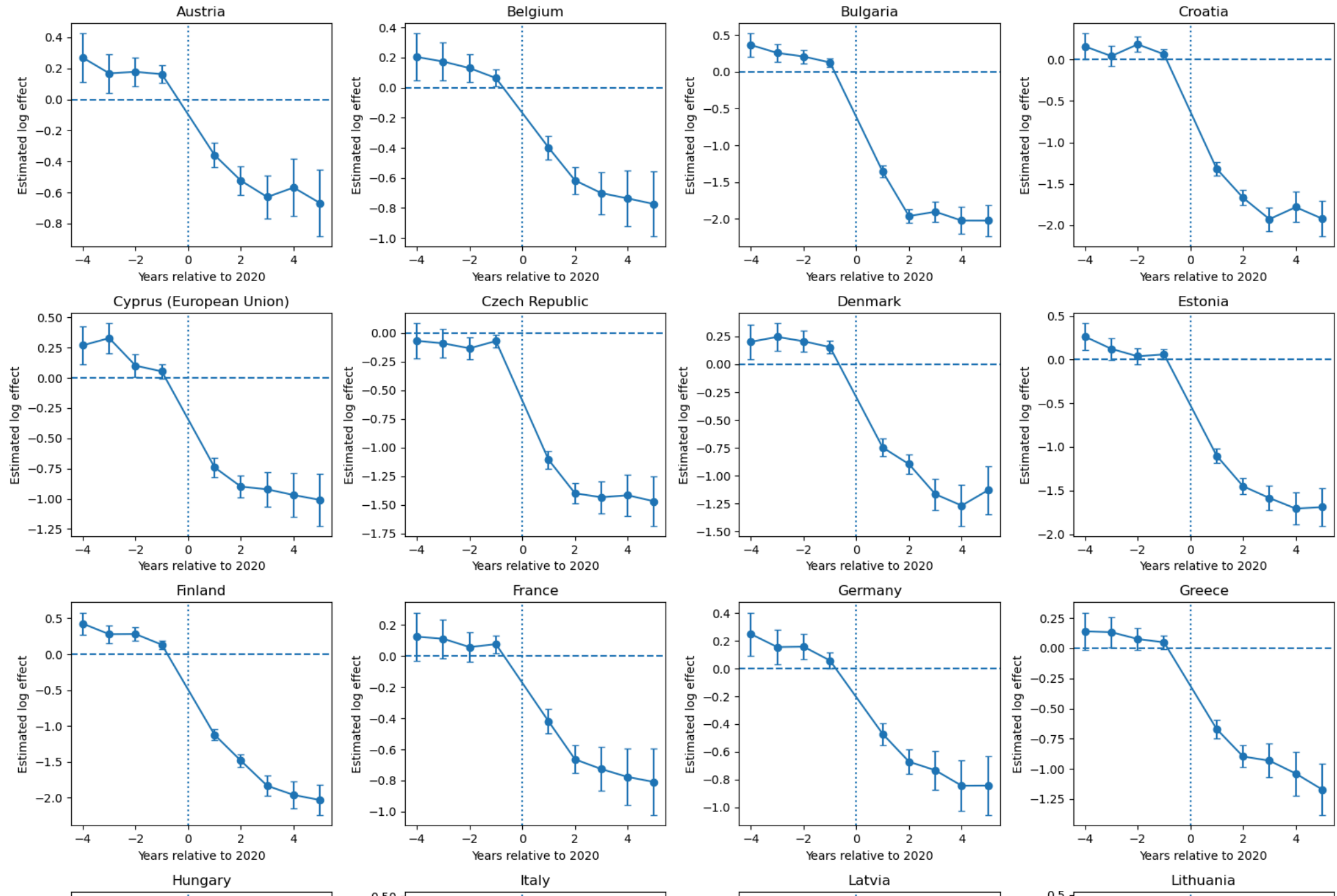
fig, axes = plt.subplots(nrows, ncols, figsize=(16, 3.5 * nrows), sharex=False, sharey=False)
axes = np.array(axes).reshape(-1)

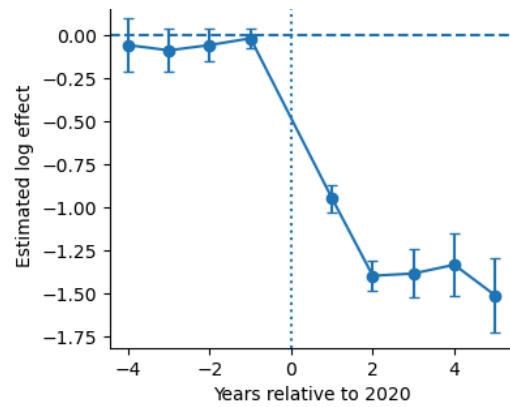
for ax, country in zip(axes, countries):
    sub = plot_sat_df[plot_sat_df["country"] == country]
    ax.axhline(0, linestyle="--")
    ax.axvline(0, linestyle=":")
    ax.errorbar(
        sub["event_time"],
        sub["estimate"],
        yerr=1.96 * sub["std_error"],
        fmt="o-",
        capsize=3
    )
    ax.set_title(country)
    ax.set_xlabel("Years relative to 2020")
    ax.set_ylabel("Estimated log effect")

# Hide unused axes
for ax in axes[len(countries):]:
    ax.set_visible(False)

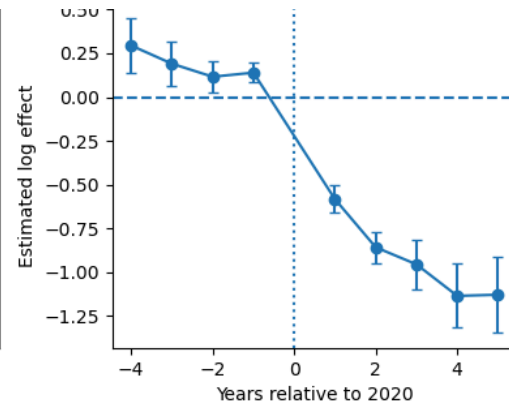
fig.suptitle("Saturated DID event study by country", y=1.02)
plt.tight_layout()
plt.show()
```

Saturated DID event study by country

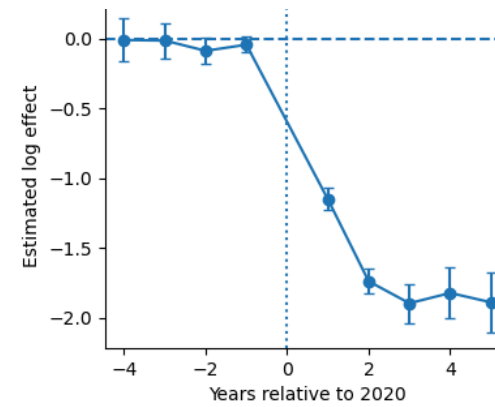




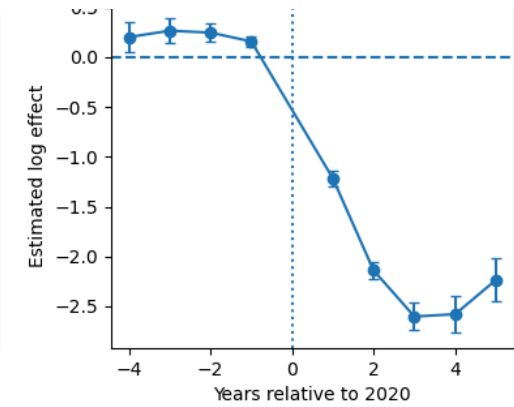
Luxembourg



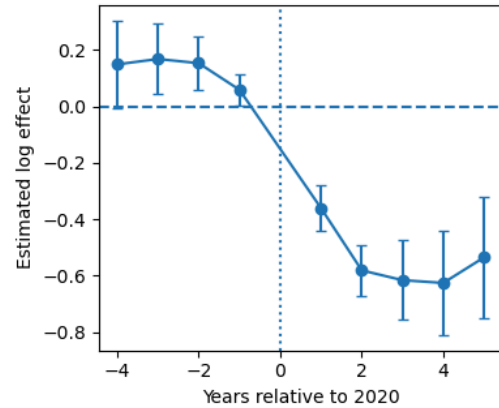
Malta



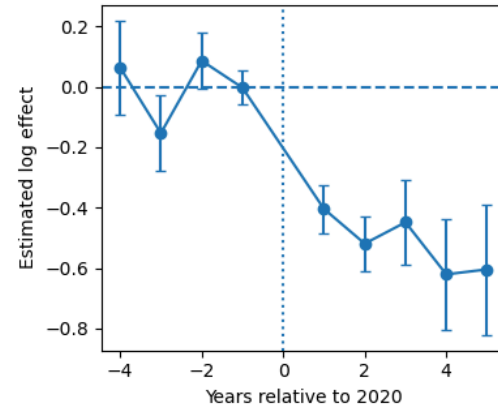
Netherlands



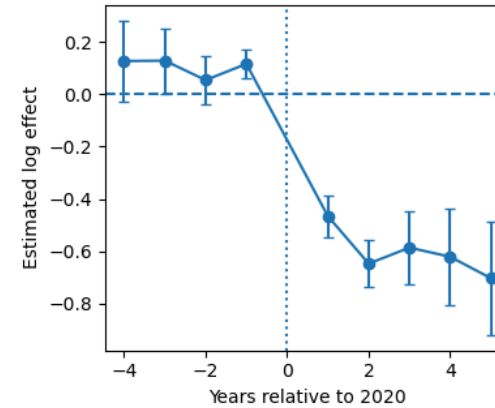
Poland



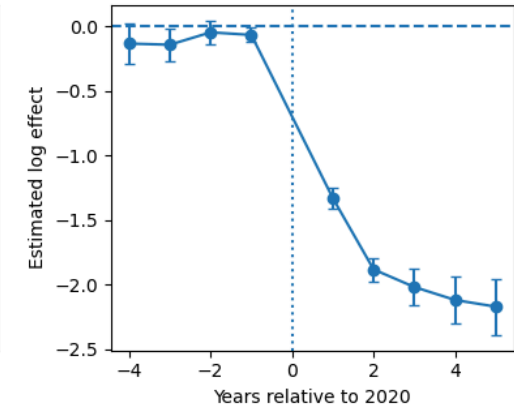
Portugal



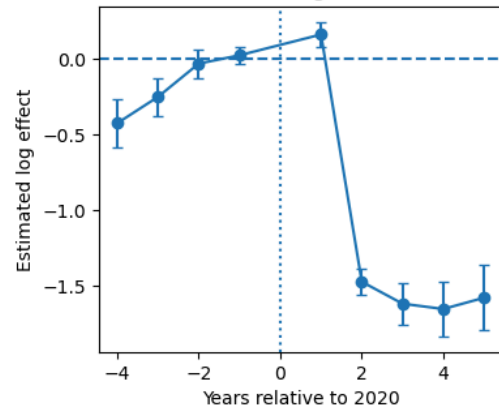
Romania



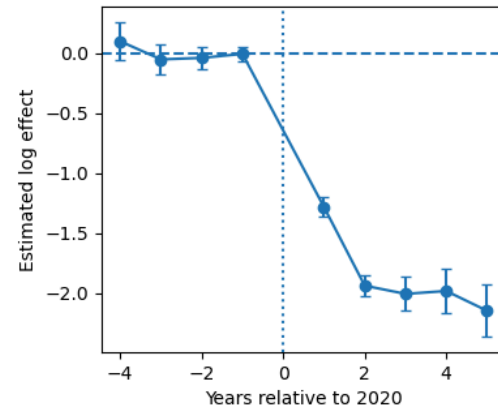
Slovakia



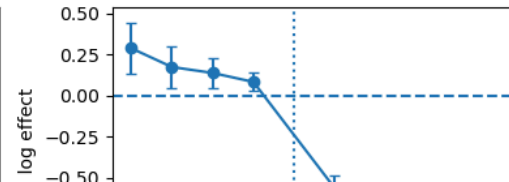
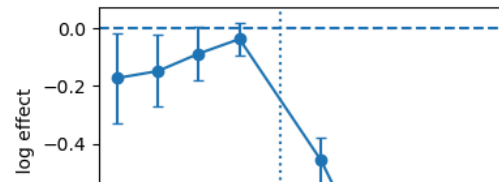
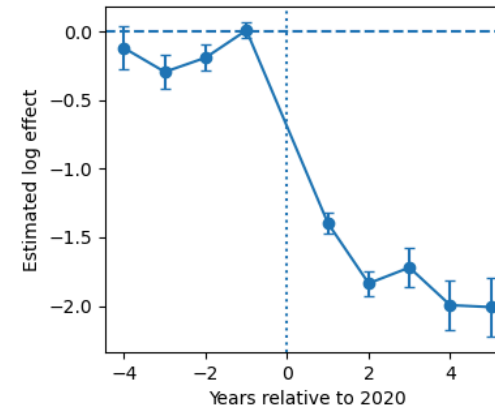
Slovenia

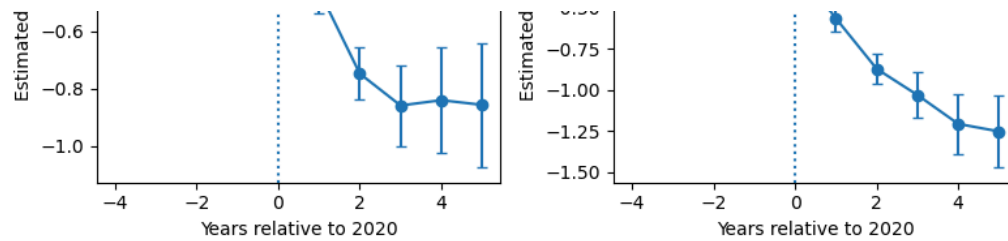


Spain



Sweden





```
In [21]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# -----
# 1. Brexit tuition shock
# -----

fee_pre = 9250
fee_post = 22200

x = np.log(fee_post / fee_pre)          # log price change
pct_fee_change = (fee_post - fee_pre) / fee_pre  # ≈ 1.40

# -----
# 2. Elasticities from DiD regression
# -----
# res_sat_DK is the fitted PanelOLS result
# from the saturated DID with country + year FE

did_df = pd.DataFrame({
    "term": res_sat_DK.params.index,
    "estimate": res_sat_DK.params.values,
    "std_error": res_sat_DK.std_errors.values
})

# Keep only treated-country coefficients
did_df = did_df[did_df["term"].str.startswith("tf_")].copy()

# Country names
did_df["country"] = did_df["term"].str.replace(r"^tf_", "", regex=True)

# 95% CI
did_df["conf.low"] = did_df["estimate"] - 1.96 * did_df["std_error"]
did_df["conf.high"] = did_df["estimate"] + 1.96 * did_df["std_error"]

# Elasticity objects
did_df["beta"] = did_df["estimate"]          # Δ Log Q
did_df["log_change"] = did_df["beta"]
did_df["conf_low_log"] = did_df["conf.low"]
did_df["conf_high_log"] = did_df["conf.high"]

did_df["pct_change_Q"] = np.exp(did_df["beta"]) - 1
did_df["eta_arc_did"] = (np.exp(did_df["beta"]) - 1) / pct_fee_change
did_df["eta_log"] = did_df["beta"] / x
```

```

did_df = did_df[[
    "country", "log_change", "conf_low_log", "conf_high_log",
    "pct_change_Q", "eta_arc_did", "eta_log"
]].copy()

print(did_df.head())

# -----
# 3. Mechanical fee-based elasticity
# -----

fee_df = (
    df[df["Domicile_named_country"].isin(eu_real)]
    .groupby("Domicile_named_country", as_index=False)
    .apply(lambda g: pd.Series({
        "app_pre": g.loc[g["Year"] == 2020, "Applicants"].mean(),
        "app_post": g.loc[g["Year"] == 2021, "Applicants"].mean()
    })))
    .reset_index(drop=True)
)

fee_df["eta_mechanical"] = (
    ((fee_df["app_post"] - fee_df["app_pre"]) / fee_df["app_pre"]) / pct_fee_change
)

fee_df = fee_df.rename(columns={"Domicile_named_country": "country"})

print(fee_df.head())

# -----
# 4. Merge datasets
# -----

plot_df = (
    did_df.merge(fee_df[["country", "eta_mechanical"]], on="country", how="left")
    .sort_values("eta_log")
    .reset_index(drop=True)
)

# small horizontal offsets so points don't overlap
plot_df["eta_arc_shift"] = plot_df["eta_arc_did"] + 0.04
plot_df["eta_mech_shift"] = plot_df["eta_mechanical"] + 0.08

# y positions for plotting
plot_df["ypos"] = np.arange(len(plot_df))

print(plot_df.head())

# -----
# 5. Plot
# -----

fig, ax = plt.subplots(figsize=(10, 12))

# zero line
ax.axvline(0, linestyle="--", linewidth=0.8)

```

```

# confidence intervals for structural elasticity
# IMPORTANT:
# In your R code, the CI plotted is still on the beta/log-change scale,
# not divided by x. To reproduce that exactly, use conf_low_log/conf_high_log.
# If you want CI for eta_log instead, divide those bounds by x.
ax.hlines(
    y=plot_df["ypos"],
    xmin=plot_df["conf_low_log"],
    xmax=plot_df["conf_high_log"],
    linewidth=1.0
)

# connectors
for _, row in plot_df.iterrows():
    ax.plot(
        [row["eta_log"], row["eta_arc_shift"]],
        [row["ypos"], row["ypos"]],
        linewidth=0.8
    )
    ax.plot(
        [row["eta_arc_shift"], row["eta_mech_shift"]],
        [row["ypos"], row["ypos"]],
        linewidth=0.8
    )

# (A) Structural elasticity
ax.scatter(
    plot_df["eta_log"],
    plot_df["ypos"],
    s=60,
    edgecolors="black",
    linewidths=0.5
)

# (B) DiD arc elasticity
ax.scatter(
    plot_df["eta_arc_shift"],
    plot_df["ypos"],
    s=55,
    edgecolors="black",
    linewidths=0.5
)

# (C) Mechanical fee elasticity
ax.scatter(
    plot_df["eta_mech_shift"],
    plot_df["ypos"],
    s=55,
    facecolors="white",
    edgecolors="black",
    linewidths=0.8
)

# Labels and Layout
ax.set_yticks(plot_df["ypos"])

```

```

ax.set_yticklabels(plot_df["country"])
ax.set_xlabel("Demand elasticity")
ax.set_ylabel("")
ax.set_title("Decomposition of Post-Brexit Demand Elasticities", fontsize=14, fontweight="bold")

# subtitle substitute
fig.text(
    0.125, 0.92,
    "Dark: structural log elasticity · Light: DiD arc elasticity · Hollow: fee-based elasticity",
    fontsize=10
)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

```

	country	log_change	conf_low_log	conf_high_log	\
0	Austria	-0.704804	-0.866481	-0.543127	
1	Belgium	-0.758927	-0.951224	-0.566630	
2	Bulgaria	-2.043416	-2.359198	-1.727633	
3	Croatia	-1.811014	-2.059726	-1.562303	
4	Cyprus (European Union)	-1.058432	-1.255379	-0.861486	

	pct_change_Q	eta_arc_did	eta_log
0	-0.505795	-0.361282	-0.805059
1	-0.531831	-0.379880	-0.866880
2	-0.870415	-0.621725	-2.334082
3	-0.836512	-0.597508	-2.068622
4	-0.653001	-0.466429	-1.208989

	country	app_pre	app_post	eta_mechanical
0	Austria	550.0	405.0	-0.188312
1	Belgium	1025.0	725.0	-0.209059
2	Bulgaria	1935.0	525.0	-0.520487
3	Croatia	355.0	100.0	-0.513078
4	Cyprus (European Union)	2655.0	1335.0	-0.355125

	country	log_change	conf_low_log	conf_high_log	pct_change_Q	\
0	Lithuania	-2.329383	-2.872416	-1.786349	-0.902644	
1	Bulgaria	-2.043416	-2.359198	-1.727633	-0.870415	
2	Finland	-1.912616	-2.380160	-1.445073	-0.852307	
3	Romania	-1.867702	-2.160074	-1.575331	-0.845522	
4	Poland	-1.824551	-2.130519	-1.518582	-0.838710	

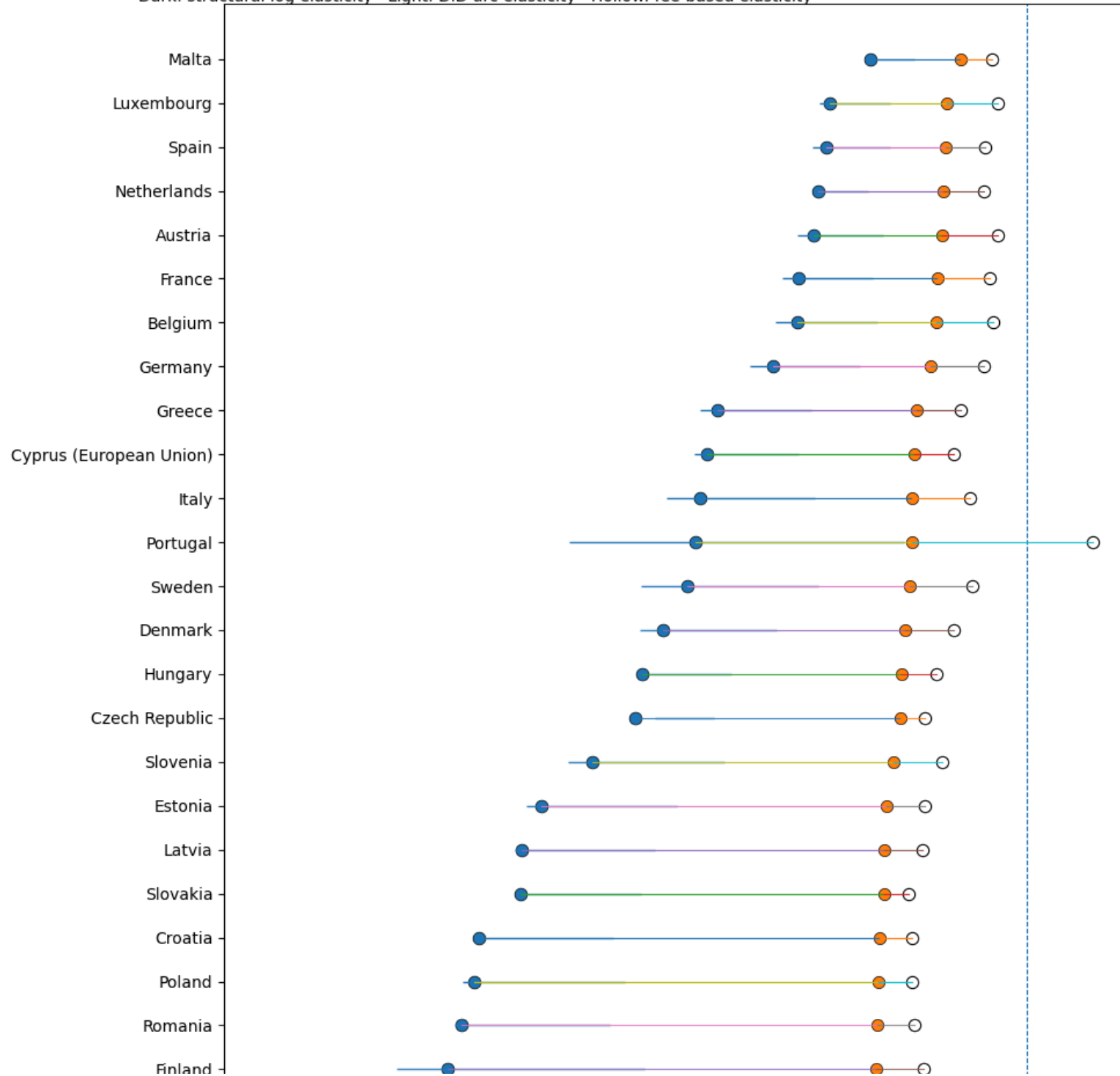
	eta_arc_did	eta_log	eta_mechanical	eta_arc_shift	eta_mech_shift	ypos
0	-0.644746	-2.660726	-0.491199	-0.604746	-0.411199	0
1	-0.621725	-2.334082	-0.520487	-0.581725	-0.440487	1
2	-0.608790	-2.184677	-0.469667	-0.568790	-0.389667	2
3	-0.603944	-2.133374	-0.505157	-0.563944	-0.425157	3
4	-0.599079	-2.084085	-0.515138	-0.559079	-0.435138	4

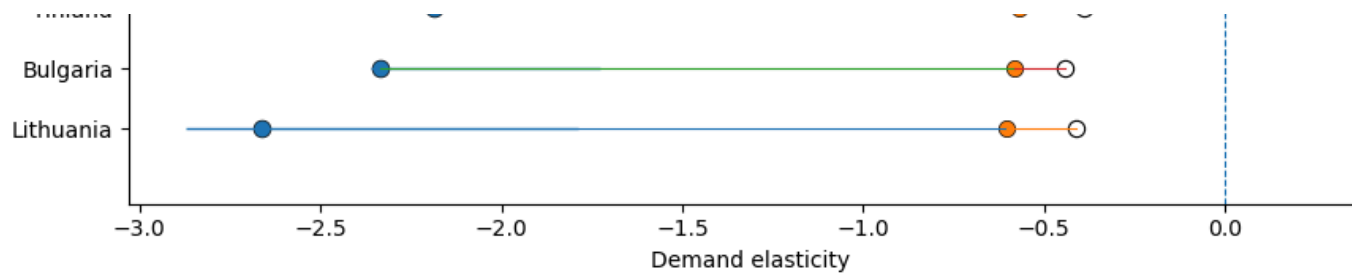
C:\Users\Stoycho.rusinov\AppData\Local\Temp\ipykernel_17000\176362530.py:61: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```
.apply(lambda g: pd.Series({
```

Decomposition of Post-Brexit Demand Elasticities

Dark: structural log elasticity · Light: DID arc elasticity · Hollow: fee-based elasticity





```
In [24]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from linearmodels.panel import PanelOLS

# =====
# S-DID design DE with COVID and Post-COVID coefficients
# =====

# -----
# 1. Prepare variables
# -----
df2 = df.copy()

df2["Domicile_named_country"] = df2["Domicile_named_country"].astype(str)
df2["Year"] = df2["Year"].astype(int)

df2["Post_COVID"] = (df2["Year"] >= 2023).astype(int)
df2["Post"] = df2["Year"].isin([2021, 2022]).astype(int)
df2["Treated"] = df2["Domicile_named_country"].isin(eu_real).astype(int)

df2["treated_factor"] = np.where(
    df2["Treated"] == 1,
    df2["Domicile_named_country"],
    "CONTROL"
)

# -----
# 2. Build interaction terms
# -----
treated_dummies = pd.get_dummies(df2["treated_factor"], prefix="tf", dtype=float)

if "tf_CONTROL" in treated_dummies.columns:
    treated_only = treated_dummies.drop(columns=["tf_CONTROL"]).copy()
else:
    treated_only = treated_dummies.copy()

# 2021-2022 interaction block
X_post = treated_only.mul(df2["Post"].to_numpy(), axis=0)
X_post.columns = [f"{col}_Post" for col in X_post.columns]

# 2023+ interaction block
X_post_covid = treated_only.mul(df2["Post_COVID"].to_numpy(), axis=0)
X_post_covid.columns = [f"{col}_Post_COVID" for col in X_post_covid.columns]
```

```

# Combine the two blocks
X_two_period = pd.concat([X_post, X_post_covid], axis=1)

# -----
# 3. Panel setup
# -----
df2_panel = df2.set_index(["Domicile_named_country", "Year"]).sort_index()
X_two_period.index = df2_panel.index
y_two_period = np.log(df2_panel["Applicants"])

# -----
# 4. Estimate model
# -----
m_sat_DK_2 = PanelOLS(
    dependent=y_two_period,
    exog=X_two_period,
    entity_effects=True,
    time_effects=True,
    drop_absorbed=True
)

res_sat_DK_2 = m_sat_DK_2.fit(cov_type="driscoll-kraay")
print(res_sat_DK_2.summary)

# -----
# 5. Extract coefficients safely
# -----
coef_long = pd.DataFrame({
    "term": res_sat_DK_2.params.index.astype(str),
    "estimate": res_sat_DK_2.params.values,
    "std_error": res_sat_DK_2.std_errors.values
})

# Keep only treated-factor coefficients
coef_long = coef_long[coef_long["term"].str.startswith("tf_").copy()]

# Create period column WITHOUT np.where
coef_long["period"] = pd.Series([None] * len(coef_long), index=coef_long.index, dtype="object")

coef_long.loc[
    coef_long["term"].str.endswith("_Post_COVID"),
    "period"
] = "Post-COVID"

coef_long.loc[
    coef_long["term"].str.endswith("_Post"),
    "period"
] = "Post-Brexit"

# Extract country name
coef_long["country"] = coef_long["term"].str.replace(r"^(tf_)", "", regex=True)
coef_long["country"] = coef_long["country"].str.replace(r"_(Post|Post_COVID)$", "", regex=True)

coef_long["elasticity"] = coef_long["estimate"]

```

```

# Keep only valid rows
coef_long = coef_long[coef_long["period"].notna()].copy()

print(coef_long.head())

# -----
# 6. Wide version for ordering and connecting lines
# -----
plot_df2 = (
    coef_long[["country", "period", "elasticity"]]
    .pivot(index="country", columns="period", values="elasticity")
    .reset_index()
)

plot_df2 = plot_df2.sort_values("Post-COVID", ascending=False).reset_index(drop=True)
plot_df2["ypos"] = np.arange(len(plot_df2))

print(plot_df2.head())

# -----
# 7. Long version for plotting points and Legend
# -----
plot_df_long = plot_df2.melt(
    id_vars=["country", "ypos"],
    value_vars=["Post-Brexit", "Post-COVID"],
    var_name="period",
    value_name="elasticity"
)

label_map = {
    "Post-Brexit": "2021 tuition shock (COVID+Brexit)",
    "Post-COVID": "Persistent response (post-COVID, Brexit only)"
}

plot_df_long["period_label"] = plot_df_long["period"].map(label_map)

print(plot_df_long.head())

# -----
# 8. Plot
# -----
fig, ax = plt.subplots(figsize=(10, 12))

# Connecting segments
for _, row in plot_df2.iterrows():
    ax.plot(
        [row["Post-Brexit"], row["Post-COVID"]],
        [row["ypos"], row["ypos"]],
        linewidth=1.0
    )

# Point layers
for label, sub in plot_df_long.groupby("period_label"):
    ax.scatter(
        sub["elasticity"],
        sub["ypos"],

```

```

        s=45,
        label=label
    )

# Zero line
ax.axvline(0, linestyle="--", linewidth=0.8)

# Axes and Labels
ax.set_yticks(plot_df2["ypos"])
ax.set_yticklabels(plot_df2["country"])
ax.set_xlabel("Demand elasticity")
ax.set_ylabel("")
ax.set_title("Change in Demand Elasticities After Brexit", fontsize=14, fontweight="bold")

# Subtitle
fig.text(
    0.125, 0.92,
    "Immediate tuition shock vs persistent post-COVID response",
    fontsize=10
)

# Legend
ax.legend(loc="upper center", bbox_to_anchor=(0.5, 1.02), frameon=False)

plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.show()

```

PanelOLS Estimation Summary

```

=====
Dep. Variable:      Applicants    R-squared:              0.8390
Estimator:          PanelOLS      R-squared (Between):    -0.0939
No. Observations:    520          R-squared (Within):     0.8335
Date:                Sat, Mar 14 2026  R-squared (Overall):    -0.0896
Time:                23:48:48      Log-likelihood          208.22
Cov. Estimator:      Driscoll-Kraay

                               F-statistic:      40.774
Entities:            52           P-value         0.0000
Avg Obs:             10.0000      Distribution:      F(52,407)
Min Obs:             10.0000
Max Obs:             10.0000      F-statistic (robust): -2.324e+16
                               P-value         1.0000
Time periods:        10          Distribution:      F(52,407)
Avg Obs:             52.000
Min Obs:             52.000
Max Obs:             52.000

```

Parameter Estimates

```

=====
Parameter   Std. Err.   T-stat   P-value   Lower CI   Upper CI
-----
tf_Austria_Post      -0.5965    0.0666   -8.9587   0.0000    -0.7274    -0.4656
tf_Belgium_Post      -0.6215    0.0779   -7.9758   0.0000    -0.7747    -0.4683
tf_Bulgaria_Post     -1.8496    0.1768  -10.463   0.0000    -2.1971    -1.5020
tf_Croatia_Post      -1.5819    0.0939  -16.840   0.0000    -1.7666    -1.3973
tf_Cyprus (European Union)_Post -0.9703    0.0936  -10.363   0.0000    -1.1544    -0.7863
tf_Czech Republic_Post -1.1814    0.0633  -18.674   0.0000    -1.3057    -1.0570
tf_Denmark_Post      -0.9828    0.0671  -14.645   0.0000    -1.1147    -0.8509
tf_Estonia_Post      -1.3731    0.1050  -13.076   0.0000    -1.5796    -1.1667
tf_Finland_Post      -1.5290    0.1379  -11.084   0.0000    -1.8002    -1.2578
tf_France_Post       -0.6160    0.0670   -9.1996   0.0000    -0.7476    -0.4843
tf_Germany_Post      -0.6959    0.0799   -8.7075   0.0000    -0.8530    -0.5388
tf_Greece_Post       -0.8640    0.0696  -12.411   0.0000    -1.0008    -0.7271
tf_Hungary_Post      -1.1286    0.0988  -11.419   0.0000    -1.3229    -0.9343
tf_Italy_Post        -0.8696    0.0930   -9.3483   0.0000    -1.0524    -0.6867
tf_Latvia_Post       -1.4134    0.1342  -10.533   0.0000    -1.6771    -1.1496
tf_Lithuania_Post    -1.8529    0.2357   -7.8619   0.0000    -2.3163    -1.3896
tf_Luxembourg_Post   -0.5764    0.0740   -7.7893   0.0000    -0.7218    -0.4309
tf_Malta_Post        -0.4602    0.0333  -13.804   0.0000    -0.5258    -0.3947
tf_Netherlands_Post  -0.6431    0.0522  -12.327   0.0000    -0.7456    -0.5405
tf_Poland_Post       -1.5271    0.1225  -12.471   0.0000    -1.7679    -1.2864
tf_Portugal_Post     -0.5191    0.3634   -1.4284   0.1539    -1.2335     0.1953
tf_Romania_Post      -1.6071    0.1528  -10.518   0.0000    -1.9075    -1.3068
tf_Slovakia_Post     -1.4976    0.0962  -15.573   0.0000    -1.6866    -1.3086
tf_Slovenia_Post     -1.2256    0.1130  -10.850   0.0000    -1.4477    -1.0036
tf_Spain_Post        -0.5146    0.0660   -7.8007   0.0000    -0.6442    -0.3849
tf_Sweden_Post       -0.8559    0.1034   -8.2748   0.0000    -1.0592    -0.6526
tf_Austria_Post_COVID -0.7770    0.0464  -16.750   0.0000    -0.8682    -0.6858
tf_Belgium_Post_COVID -0.8505    0.0478  -17.784   0.0000    -0.9446    -0.7565
tf_Bulgaria_Post_COVID -2.1727    0.0777  -27.961   0.0000    -2.3254    -2.0199
tf_Croatia_Post_COVID -1.9637    0.0330  -59.436   0.0000    -2.0287    -1.8988
tf_Cyprus (European Union)_Post_COVID -1.1172    0.0781  -14.305   0.0000    -1.2707    -0.9637
tf_Czech Republic_Post_COVID -1.3682    0.0215  -63.497   0.0000    -1.4106    -1.3259
tf_Denmark_Post_COVID -1.3494    0.0509  -26.512   0.0000    -1.4495    -1.2494

```

tf_Estonia_Post_COVID	-1.7574	0.0565	-31.081	0.0000	-1.8686	-1.6463
tf_Finland_Post_COVID	-2.1684	0.0953	-22.765	0.0000	-2.3556	-1.9811
tf_France_Post_COVID	-0.8450	0.0302	-28.026	0.0000	-0.9043	-0.7857
tf_Germany_Post_COVID	-0.9324	0.0570	-16.368	0.0000	-1.0443	-0.8204
tf_Greece_Post_COVID	-1.1274	0.0611	-18.443	0.0000	-1.2475	-1.0072
tf_Hungary_Post_COVID	-1.3655	0.0368	-37.066	0.0000	-1.4379	-1.2931
tf_Italy_Post_COVID	-1.2225	0.0664	-18.424	0.0000	-1.3529	-1.0920
tf_Latvia_Post_COVID	-1.8374	0.0154	-119.12	0.0000	-1.8677	-1.8071
tf_Lithuania_Post_COVID	-2.6470	0.0976	-27.118	0.0000	-2.8389	-2.4551
tf_Luxembourg_Post_COVID	-0.6977	0.0426	-16.368	0.0000	-0.7815	-0.6139
tf_Malta_Post_COVID	-0.5564	0.0417	-13.330	0.0000	-0.6385	-0.4744
tf_Netherlands_Post_COVID	-0.7233	0.0347	-20.850	0.0000	-0.7915	-0.6552
tf_Poland_Post_COVID	-2.0228	0.0452	-44.792	0.0000	-2.1116	-1.9341
tf_Portugal_Post_COVID	-1.4797	0.1041	-14.218	0.0000	-1.6842	-1.2751
tf_Romania_Post_COVID	-2.0414	0.0391	-52.276	0.0000	-2.1182	-1.9646
tf_Slovakia_Post_COVID	-1.7893	0.0889	-20.122	0.0000	-1.9641	-1.6145
tf_Slovenia_Post_COVID	-1.5760	0.1029	-15.313	0.0000	-1.7783	-1.3736
tf_Spain_Post_COVID	-0.7636	0.0404	-18.921	0.0000	-0.8429	-0.6842
tf_Sweden_Post_COVID	-1.3013	0.0748	-17.400	0.0000	-1.4483	-1.1543

F-test for Poolability: 254.06

P-value: 0.0000

Distribution: F(60,407)

Included effects: Entity, Time

	term	estimate	std_error	period \
0	tf_Austria_Post	-0.596481	0.066581	Post-Brexit
1	tf_Belgium_Post	-0.621494	0.077923	Post-Brexit
2	tf_Bulgaria_Post	-1.849559	0.176777	Post-Brexit
3	tf_Croatia_Post	-1.581933	0.093938	Post-Brexit
4	tf_Cyprus (European Union)_Post	-0.970308	0.093628	Post-Brexit

	country	elasticity
0	Austria	-0.596481
1	Belgium	-0.621494
2	Bulgaria	-1.849559
3	Croatia	-1.581933
4	Cyprus (European Union)	-0.970308

period	country	Post-Brexit	Post-COVID	ypos
0	Malta	-0.460216	-0.556437	0
1	Luxembourg	-0.576376	-0.697690	1
2	Netherlands	-0.643082	-0.723349	2
3	Spain	-0.514575	-0.763567	3
4	Austria	-0.596481	-0.777019	4

	country	ypos	period	elasticity \
0	Malta	0	Post-Brexit	-0.460216
1	Luxembourg	1	Post-Brexit	-0.576376
2	Netherlands	2	Post-Brexit	-0.643082
3	Spain	3	Post-Brexit	-0.514575
4	Austria	4	Post-Brexit	-0.596481

	period_label
0	2021 tuition shock (COVID+Brexit)
1	2021 tuition shock (COVID+Brexit)
2	2021 tuition shock (COVID+Brexit)

3 2021 tuition shock (COVID+Brexit)
4 2021 tuition shock (COVID+Brexit)

Change in Demand Elasticities After Brexit

Immediate tuition shock vs persistent post-Covid tuition shock (COVID+Brexit)

● Persistent response (post-COVID, Brexit only)

